

Justificación y explicación del Diseño

ECSDI

Q2 2025-26

Grupo 11

Jaume López

Genís López

Ramon Sánchez



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

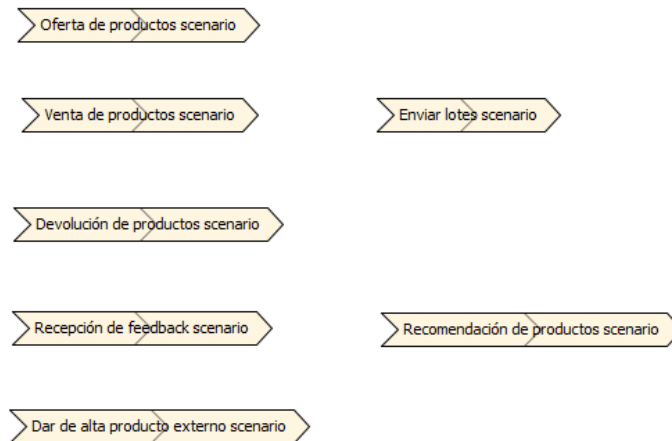


Tabla de contenidos

1. Especificación del sistema.....	2
1.1 Escenarios.....	2
1.2 Objetivos.....	4
1.3 Roles.....	5
1.4 Visión general.....	7
2. Diseño arquitectónico.....	8
2.1 Agrupación de datos.....	8
2.2 Agrupación de roles en agentes.....	10
2.3 Interacción entre agentes.....	11
2.4 Visión general.....	12
3. Diseño Detallado.....	14
3.1. Agente Valorador.....	14
3.2. Ag. Catalogador.....	17
3.3. Ag. Agente lógístico.....	18
3.4. Ag. Ventas.....	21
3.5. Ag. Tesorero.....	24
4. Ontología.....	27
4.1. Conceptos.....	28
4.2. Atributos.....	32
4.3. Relaciones.....	37
5. Explicación y justificación de la implementación en python.....	41
5.1 Arquitectura general de la implementación.....	41
5.2 Interfaz de cliente y experiencia de demostración.....	42
5.3 Implementación de los agentes internos.....	42
5.4 Implementación de actores externos.....	44
5.5 Protocolos, trazabilidad y temporizadores.....	44
5.6 Justificación de las decisiones técnicas.....	45
6. Descripción detallada de la planificación del trabajo.....	46
6.1 Canales de Comunicación.....	46
6.2 Planificación y Estrategia de Desarrollo por Componentes.....	47
7. Evaluación crítica de los resultados.....	48
7.1. Consideraciones sobre los límites de la solución y áreas de mejora.....	48
7.2. Elementos incluidos del nivel avanzado.....	49

1. Especificación del sistema

1.1 Escenarios



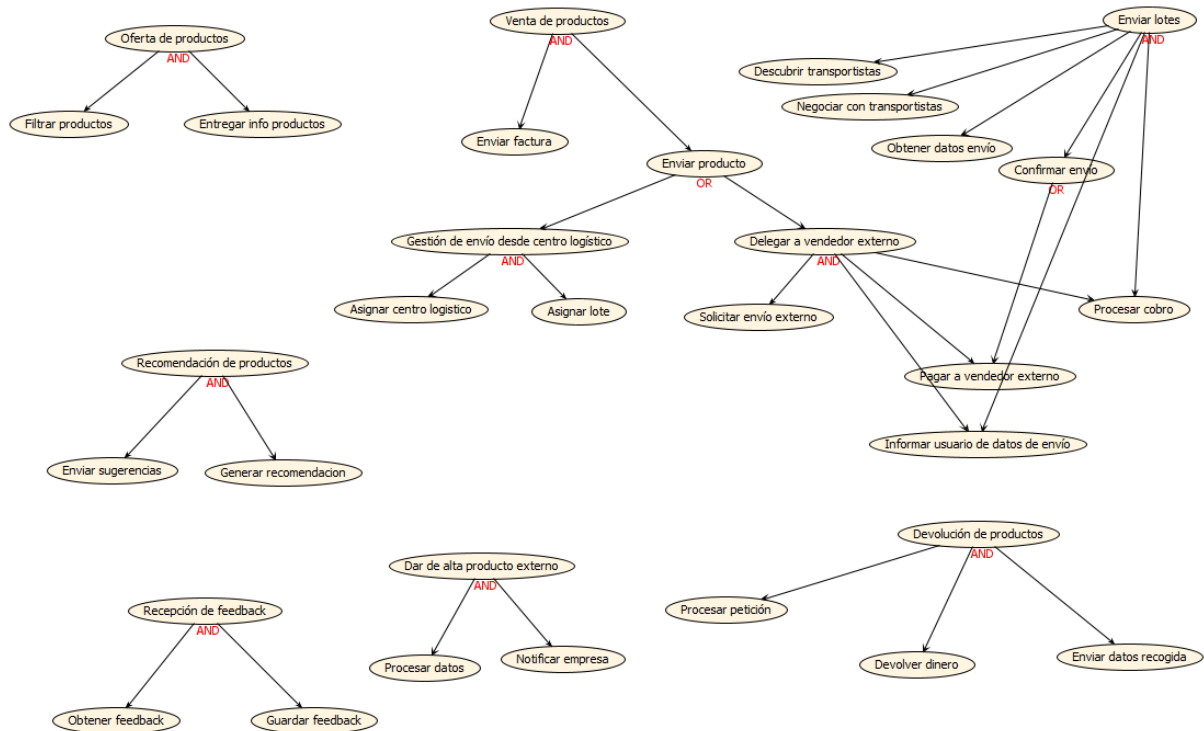
Lo primero que hicimos fue especificar los escenarios. En un primer momento, definimos cada uno de los requisitos que aparecen en el documento de explicación de la práctica como escenarios, los cuales son “Oferta de productos”, “Venta de productos”, “Devolución de productos”, “Recepción de feedback” y “Recomendación de productos”.

Más adelante, nos dimos cuenta de un escenario extra, el cual no sale especificado como tal pero que podemos intuir que existe, el cual es “Dar de alta producto externo”. Este escenario es el que se desarrolla cuando una empresa externa quiere dar de alta un producto dentro de nuestra tienda, lo cual es una funcionalidad que se nos pide.

Finalmente, mientras desarrollamos los objetivos de venta de productos, vimos que era necesario separar parte de este en otro escenario “Enviar lotes”. Esto es así, ya que aunque de primeras pudiera parecer un mismo escenario, nos dimos cuenta de que solo una parte de todo el proceso que se desarrolla desde que un usuario pide un producto hasta que le llega a casa puede ser realizada al momento, y que intervienen demasiadas variables.

En concreto, el escenario “Venta de productos”, engloba todas las operaciones que se encargan de asignar un centro logístico y que este asigne la comanda a un lote. Más tarde, cuando sea activado el escenario “Enviar lotes” mediante un temporizador que se ejecutará ciertas veces al día, empezará todo el proceso de buscar un transportista y notificar al usuario una vez un transportista haya sido asignado, además de pagar a una empresa externa si el producto pedido era suyo.

1.2 Objetivos



A la par que describimos los escenarios fuimos describiendo los objetivos, ya que son la pieza fundamental que nos permitió más adelante descubrir los diferentes roles, y en consecuencia los agentes y bases de datos.

En un primer momento fue todo bastante sobre la marcha, sin mucha complicación, pero a medida que iba avanzando nos dimos cuenta de cosas que no nos cuadraban, como por ejemplo lo que se ha mencionado anteriormente relacionado con los escenarios de “Venta de productos” y “Enviar lotes”.

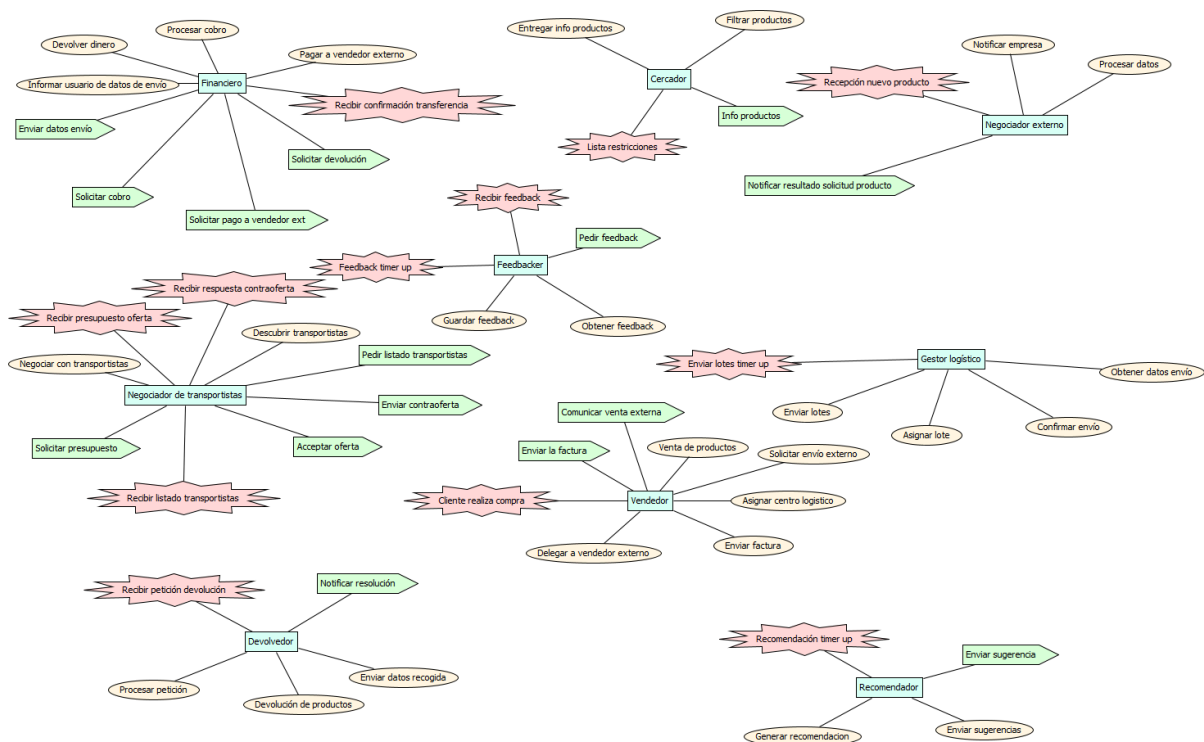
Hay tres objetivos que son compartidos: “Pagar a vendedor externo”, “Procesar cobro” y “Informar usuario de datos de envío”. Esto es así ya que, en caso de que el producto que el usuario ha pedido sea totalmente externo, es decir, que sea de una empresa externa y además no lo tengamos en ningún centro logístico, hay que pagar a la empresa externa, cobrar al usuario y notificar al usuario.

El objetivo de pagar al vendedor externo y procesar cobro también existe en el caso de que el producto sea de una empresa externa pero sí que lo tengamos en nuestros almacenes, en cuyo caso habrá que pagar a la empresa externa y cobrar al usuario sólo cuando se

haya asignado un transportista a el lote que lo contiene, es por eso que forma parte de dos objetivos padre diferentes.

El objetivo de notificar al usuario es compartido ya que, a parte del primer caso comentado, también hay que notificar al usuario cuando un transportista ha sido asignado para transportar su paquete.

1.3 Roles



Una vez finalizados los objetivos, ya pudimos agruparlos en distintos roles.

Finalmente obtuvimos los siguientes roles:

- 1) **Financiero:** Rol encargado de interaccionar con la entidad bancaria, gestionar pagos y enviar los datos d'envío al usuario cobrado.
- 2) **Cercador:** Rol encargado de buscar dentro del catálogo de productos aquellos productos que se le pidan o que cumplan ciertos filtros.
- 3) **Feedbacker:** Rol encargado de gestionar y pedir feedback a los clientes que han comprado productos.
- 4) **Negociador de transportistas:** Rol encargado de interactuar con los transportistas para repartir los lotes de un centro.
- 5) **Vendedor:** Rol encargado de gestionar el proceso de venta de unos productos

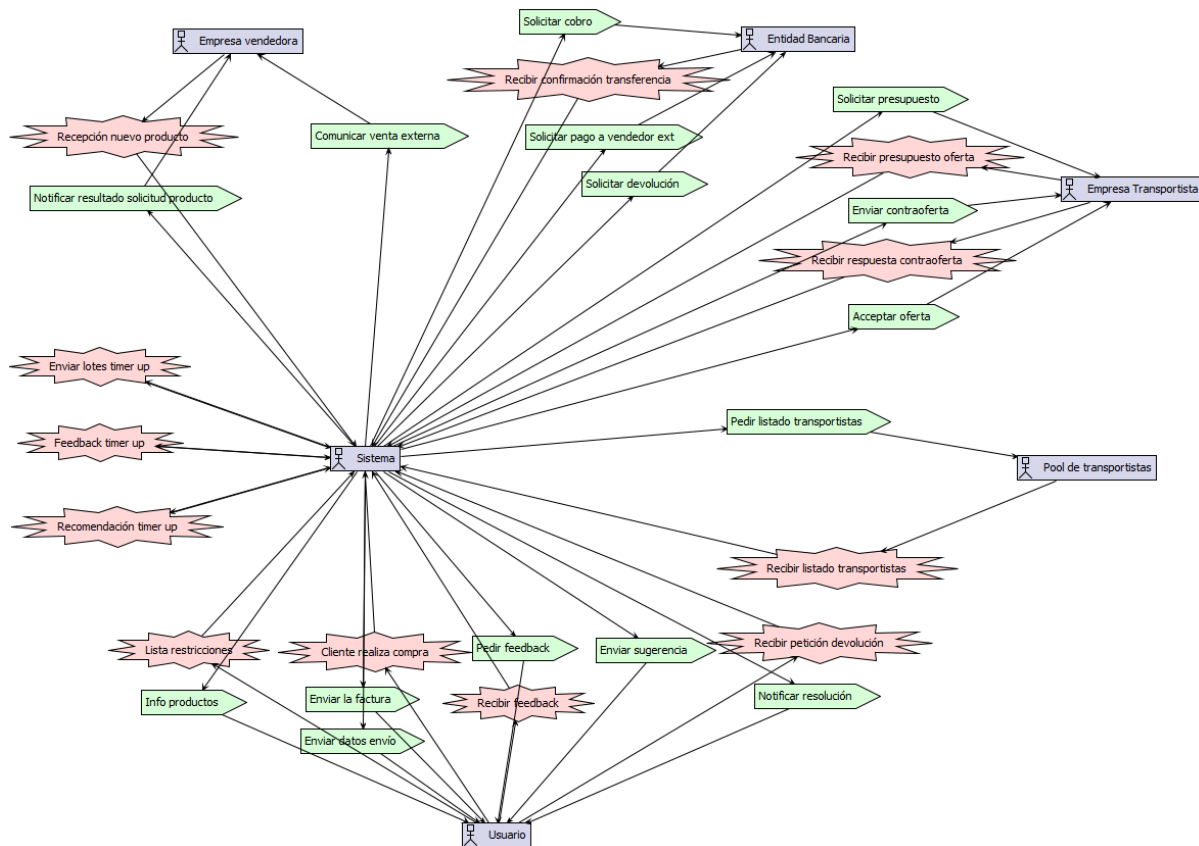
- 6) **Devolvedor:** Rol encargado de gestionar devoluciones
- 7) **Recomendador:** Rol encargado de generar y enviar recomendaciones personalizadas a los usuarios.
- 8) **Gestor Logístico:** Rol encargado de gestionar las entradas y salidas de stock y lotes de un centro logístico.
- 9) **Negociador externo:** Rol encargado de interactuar con las empresas vendedoras de productos.

Una decisión muy importante que tomamos fue definir el rol de gestor logístico, ya que nos dimos cuenta que era necesario para poder representar la idea de que podemos tener varios centros logísticos. Es por esto que parte del escenario de la venta se ve distribuido, ya que llega un momento que la competencia pasa a ser de un centro logístico. Aquí también interviene el rol de negociador de transportistas, que es el encargado de descubrir y asignar (negociando) transportistas a los lotes del centro.

También decidimos hacer que todas las operaciones bancarias las realizara el rol Financiero. Este es el que mediante la acción “Solicitar cobro” pedirá a la entidad bancaria externa que realice los cobros, “Solicitar devolución” que pedirá la devolución de dinero cobrado previamente y “Solicitar pago a vendedor ext” para pagar a las tiendas externas. Esta responde con la percepción “Recibir confirmación transferencia”, que en el contexto de la práctica siempre va a ser positiva ya que no se ha especificado que pueda pasar lo contrario, pero decidimos representarlo. Después de confirmar la transferencia de una compra el financiero mismo será el encargado de enviar los datos de envío de esa compra al usuario.

Otra decisión parecida ha sido la de especificar el rol “Negociador externo”, encargado de comunicarse con las empresas de productos externos para incluir nuevos productos en el catálogo de nuestra tienda, pero no para pedir a una empresa externa que realice el reparto de un producto que un cliente ha pedido mediante nuestra tienda, pero que no tenemos en ningún centro logístico, este objetivo lo gestionara el vendedor de una forma parecida a una compra normal, donde se avisa a la empresa externa en lugar de a un centro logístico.

1.4 Visión general



A continuación, decidimos cuáles serían los actores externos con los que nuestro sistema interactúa, y cuáles son las percepciones y acciones que corresponden a cada uno de ellos.

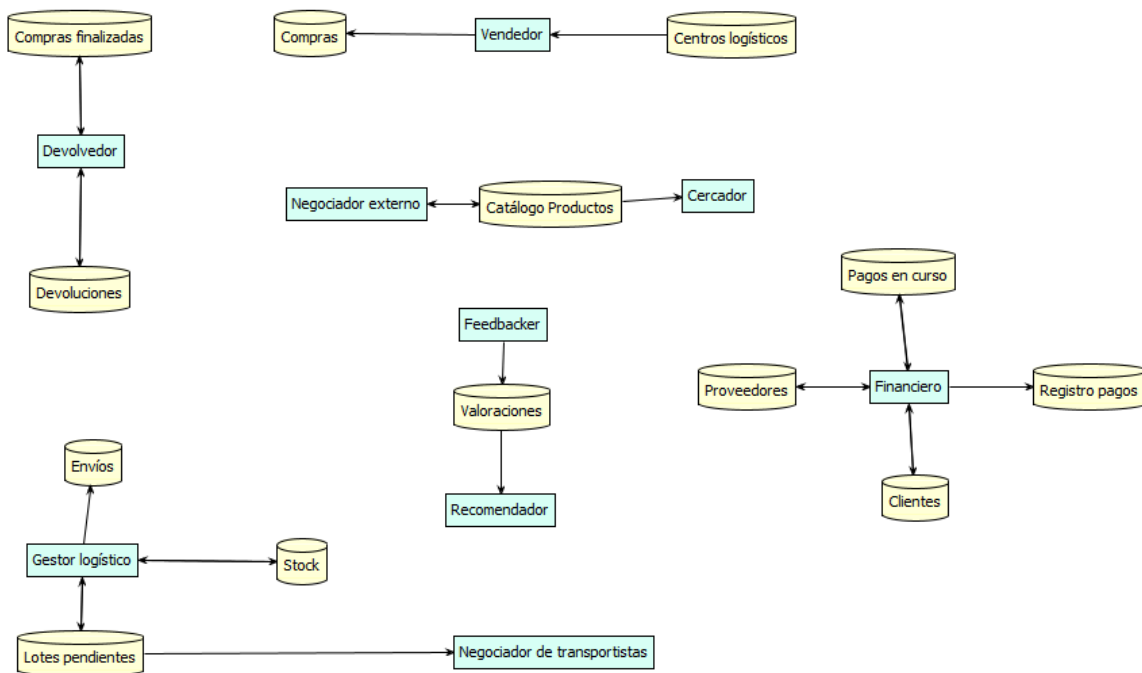
Algunos como Usuario, Empresa vendedora y empresa Transportista estaban muy claros desde un principio. En cambio, Pool de transportistas y Entidad Bancaria fueron actores que surgieron poco a poco a medida que intentamos solucionar cada requisito del enunciado.

- 1) **Pool de transportistas** es el actor que representa el sistema que nos ofrece la disponibilidad de transportistas de ese momento.
- 2) **Entidad Bancaria** representa el sistema externo que nos va a permitir realizar transacciones.

En general fue bastante sobre la marcha, ya que fue muy sencillo distinguir a quien iban dirigidas las acciones y percepciones realizadas por el sistema.

2. Diseño arquitectónico

2.1 Agrupación de datos



Para la agrupación de datos, nos centramos en qué tipos de datos tenemos para saber quién los va a leer o modificar, con eso en mente obtuvimos las siguientes agrupaciones de datos:

- 1) **Compras:** Contiene toda la información relacionada con las compras realizadas que aun no se han entregado (producto, importe, fecha de la compra, comprador). Estos datos serán gestionados por el vendedor, que es el que se encarga de crear cada instancia.
- 2) **Compras finalizadas:** Contiene toda la información relacionada con compras que ya se han entregado y pagado (producto, importe, fecha de la compra, comprador). A diferencia de Compras las compras finalizadas solo le interesan a Devolvedor y a Feedbacker, decidimos que tenía más sentido que Devolvedor las gestionase ya que es el que determina qué información queremos guardar de las compras finalizadas.
- 3) **Catálogo Productos:** Contiene toda la información relacionada con los productos (metadatos del producto, descripción, precios, si es externo, interno, híbrido, etc.) actualmente solo podría añadir productos a esta el Negociador externo, pero como

Cercador tendra muchisimos accesos es mas eficiente que este tambien sea responsable de estos datos.

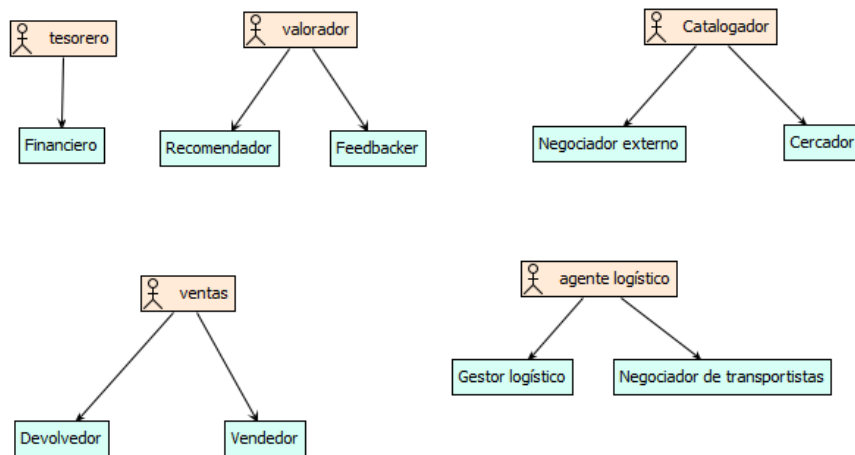
- 4) **Valoraciones:** Contiene las valoraciones de los productos, que los clientes han escrito. La informacion y condiciones de las valoraciones son determinadas por el feedbacker, por lo que este sera el responsable.
- 5) **Devoluciones:** Contiene información relevante sobre la devolución de una compra, especialmente el motivo. El Devolvedor es el claro responsable de estos datos que de momento solo accede el.
- 6) **Centros logísticos:** Contiene el listado de los centros logísticos internos. El vendedor es el unico interesado en conocer los centros logísticos y como estos tampoco se han de poder modificar el sera el responsable de estos datos.
- 7) **Envíos:** Contiene toda la información relacionada con los envíos en proceso. Los datos de los envios los conoce el gestor logístico y solo le interesan a el, asi que este sera el responsable.
- 8) **Stock:** Contiene toda la información relacionada con el stock de productos / existencias (cantidad, etc.). Como estos datos son informacion de cada centro logistico el responsable sera su gestor logistico.
- 9) **Lotes pendientes:** Contiene toda la información relacionada con los lotes de productos (destino, peso, fecha de entrega, prioridad, etc..). Estos datos son creados por cada gestor logistico pero tambien son necesarios para el Negociador de transportistas.
- 10) **Pagos en Curso & Registro pagos:** Aunque aparentemente estos no sean necesarios para el funcionamiento del sistema sí que los requerimos para poder gestionar correctamente los cobros en espera y finalizados, lo cual es un proceso clave del sistema.
- 11) **Clientes & Proveedores(Tiendas externas):** Estas tablas son claves para delegar todos los procesos de pago al financiero sin tener que arrastrar todos los datos de los clientes o de los proveedores con el fin de cobrar o pagarles al finalizar la confirmación del envío de una compra.

Con el enunciado nos dimos cuenta de que era necesario tener una agrupación de nuestros centros logísticos, la cual solo será accedida por Vendedor ya que no se nos pide la intención de poder modificar los centros logísticos.

Decidimos separar el Catálogo Productos, que especifica la información genérica de cada producto, del Stock, que contiene el Stock de un centro logístico, para permitir al gestor logístico de cada centro logístico trabajar solo con los datos de su Stock.

También nos encontramos con el inconveniente de que no llegamos a la conclusión de requerir los datos de envíos anteriores en algún escenario, sin embargo, al poseer estos datos concluimos que era conveniente guardarlos para su posible posterior uso.

2.2 Agrupación de roles en agentes



Para agrupar los roles no nos encontramos con ninguna complicación, ya que teníamos claro que el objetivo era aislar los roles según las responsabilidades y datos que comparten. Lo primero y más claro eran los roles Gestor logístico y Negociador de transportistas, que podían ser realizados por cada centro logístico de forma independiente al resto de procesos.

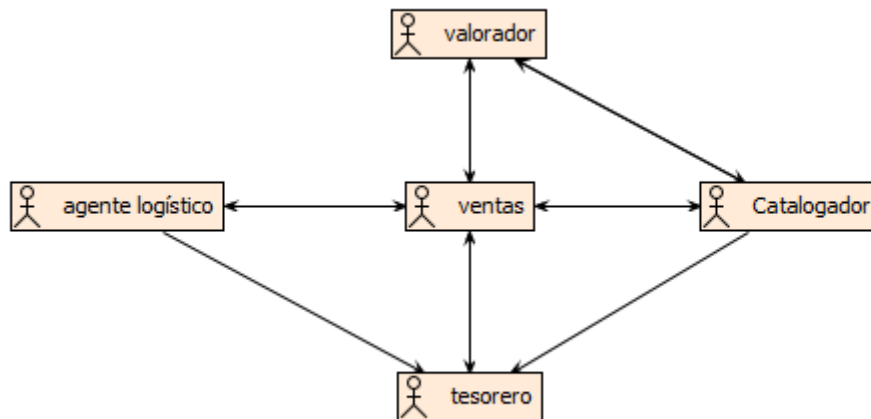
También resultó lógico agrupar los roles de Recomendador y Feedbacker, debido a que ejecutan trabajos similares y comparten los datos de valoraciones.

Posteriormente, decidimos agrupar Devolvedor y Vendedor porque todos ellos forman parte del sistema central de compras de la tienda, el Vendedor se encarga de crear las compras mientras que el devolvedor las almacena a la espera de una posible devolución.

Por último agrupamos los roles de Cercador y Negociador externo, que son los encargados de gestionar operaciones sobre el catálogo de productos.

Con estos agrupamientos solo quedo un rol sin agrupar que es el Financiero, que tiene unas responsabilidades independientes de los procesos de los otros roles, lo que nos llevó a la conclusión de que podía ser un agente con solo ese rol.

2.3 Interacción entre agentes



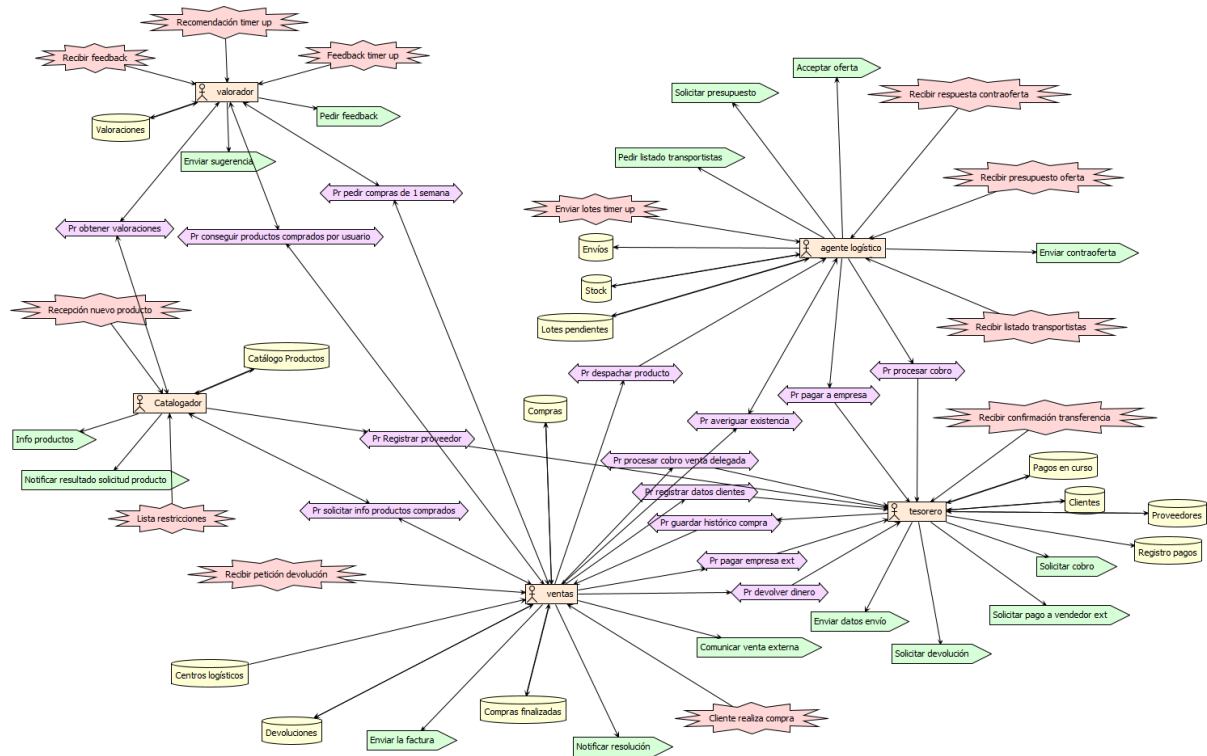
Por lo que respecta a los agentes, podemos observar que el agente valorador interactúa con los agentes del sistema Ventas y Catalogador, esto se debe a que Valorador para completar las tareas del rol de feedbacker necesita pedirle a ventas información sobre las compras anteriores, y Catalogador le pedirá la información de las valoraciones para mostrarlas.

Por otro lado, el agente de tesorero recibe peticiones de los otros agentes, porque casi todos los sistemas necesitan de algún proceso, comprobación o interacción con la entidad bancaria (ya sea para hacer pagos o recibirlos). Hay que tener en cuenta que el tesorero solo recibe peticiones, pero no responde a ellas, porque en el marco de nuestro proyecto, las solicitudes de pago hechas a la entidad bancaria siempre son satisfactorias; por ende, no contemplamos el caso de fallo de este.

Asimismo, las ventas sí que interaccionan bidireccionalmente como el agente logístico, ya que deben intercambiarse los productos existentes, y comunicar el despacho de un producto (asignarle lote a este).

Por último, el agente Catalogador, recibe la comunicación de solicitud de datos de productos de una venta, este también deberá notificar al tesorero de los datos nuevos o actualizados de las tiendas externas.

2.4 Visión general



En este último diagrama podemos observar a todo color la distribución de nuestro sistema, con sus bases de datos, protocolos (y mensajes internos), así como las percepciones y acciones.

También hemos añadido en esta sección los protocolos con sus correspondientes mensajes intercambiados:

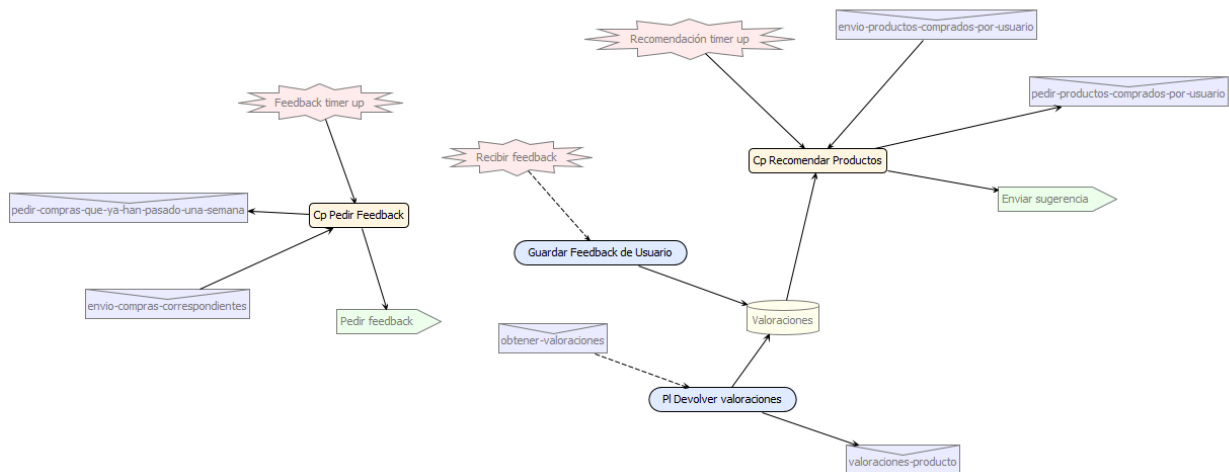
- 1) **Pr procesar cobro:** Gestiona el pago de una compra; ventas pide al tesorero cobrar al cliente y confirmar la transferencia.
- 2) **Pr devolver dinero:** Permite reembolsar al cliente tras aceptar una devolución, mediante una transferencia gestionada por el tesorero.
- 3) **Pr despachar producto:** Coordina la preparación del envío; ventas pide al agente logístico asignar el producto a un lote.
- 4) **Pr averiguar existencia:** Sirve para comprobar si hay stock del producto consultando al agente logístico.

- 5) **Pr solicitar info productos comprados:** Se usa para obtener los datos de un producto comprado como su precio, proveedor...
- 6) **Pr pagar empresa ext:** Gestiona el pago a una empresa externa para que envíe el producto comprado.
- 7) **Pr pagar a empresa:** Sirve para pagar el valor del producto al proveedor correspondiente tras preparar el envío.
- 8) **Pr obtener valoraciones:** Permite al Catalogador solicitar las valoraciones de ciertos productos al Valorador.
- 9) **Pr conseguir productos comprados por usuario:** Permite al Valorador pedir al Vendedor información de productos comprados anteriormente por un Usuario para generar una recomendación más acertada
- 10) **Pr pedir compras de 1 semana:** El Feedbacker necesita esta información para poder solicitar el feedback de productos con un margen lógico.
- 11) **Pr procesar cobro venta delegada:** Se cobra al Usuario la venta de un producto que se ha delegado el envío a la tienda externa que es dueña del producto.
- 12) **Pr guardar historico compra:** El Financiero notifica al vendedor que compras se han cobrado para poder marcarlas como finalizadas.
- 13) **Pr Registrar proveedor:** Se notifica de los datos de un proveedor al financiero para poder realizarles un pago en el futuro.
- 14) **Pr registrar datos clientes:** Se notifica de los datos de un cliente al financiero para poder realizarles un cobro en el futuro.

En resumen, los agentes se interrelacionan con protocolos e intercambiando mensajes y cada uno guarda la información pertinente a su ámbito. Cabe destacar que diseñamos el agente Gestor Logístico para representar una instancia de un Centro Logístico, teniendo un conjunto de estos, y tenerlos almacenados dentro de la base de datos *Centros Logísticos*.

3. Diseño Detallado

3.1. Agente Valorador



El agente Valorador es el encargado de gestionar todo el ciclo de valoraciones y recomendaciones dentro del sistema. Se ocupa tanto de solicitar feedback a los usuarios como de almacenarlo y utilizarlo posteriormente para generar sugerencias personalizadas. El agente se estructura en dos capacidades principales: Pedir Feedback y Valoraciones/Recomendaciones, además de varios planes que coordinan el comportamiento global.

Plan Guardar Feedback de Usuario: Este plan se activa cuando el agente recibe la percepción Recibir feedback, es decir, cuando un usuario ha enviado una valoración.

El plan se encarga de almacenar esta información en el recurso Valoraciones, asegurando que los datos queden persistidos para su uso futuro.

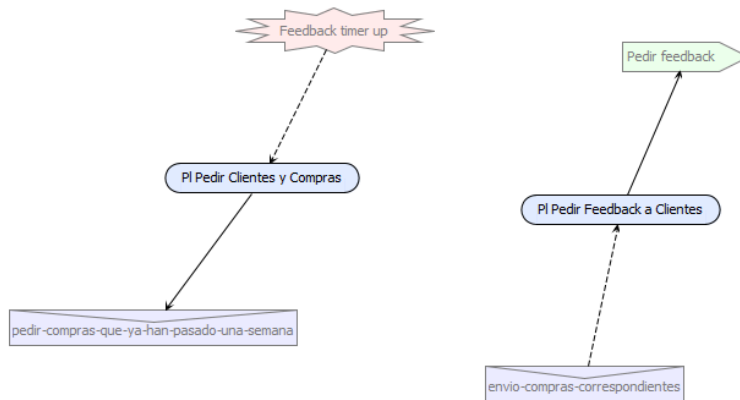
Su función es mantener actualizada la base de datos de valoraciones del sistema.

Plan Devolver valoraciones: Este plan se activa cuando otro agente solicita información mediante el mensaje obtener-valoraciones.

El agente consulta el recurso Valoraciones y devuelve los datos correspondientes a través del mensaje valoraciones-producto.

Este plan permite que otros agentes del sistema utilicen las valoraciones para enriquecer su propia lógica.

3.1.1. Cp Pedir Feedback

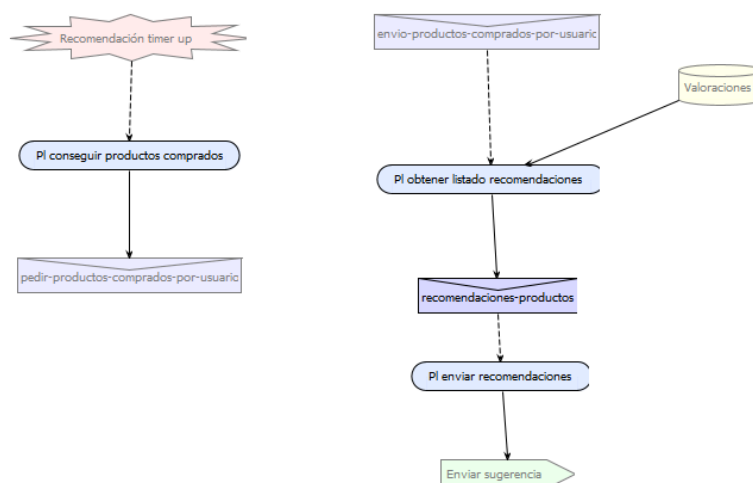


Esta capacidad agrupa la lógica necesaria para identificar a qué usuarios se debe solicitar feedback y realizar dicha petición.

Plan Pedir Clientes y Compras: Este plan se activa con la percepción Feedback timer up. Se encarga de solicitar las compras que cumplen la condición temporal (más de una semana) mediante el mensaje pedir-compras-que-ya-han-pasado-una-semana. Su objetivo es obtener la información necesaria para saber a qué usuarios contactar.

Plan Pedir Feedback a Clientes: Este plan se activa cuando se recibe el mensaje envio-compras-correspondientes con las compras filtradas. A partir de esta información, el plan envía solicitudes de feedback a los usuarios mediante la acción Pedir feedback. Su función es completar el proceso iniciando la interacción directa con el usuario.

3.1.2. Cp Recomendar Productos



Esta capacidad contiene la lógica relacionada con el uso de datos para generar valor, tanto almacenando valoraciones como creando recomendaciones.

Plan Conseguir productos comprados Este plan se activa con la percepción Recomendación timer up.

Solicita el historial de compras de los usuarios mediante el mensaje pedir-productos-comprados-por-usuario, que será necesario para generar recomendaciones.

Plan Obtener listado de recomendaciones: Este plan se activa al recibir el mensaje envio-productos-comprados-por-usuario.

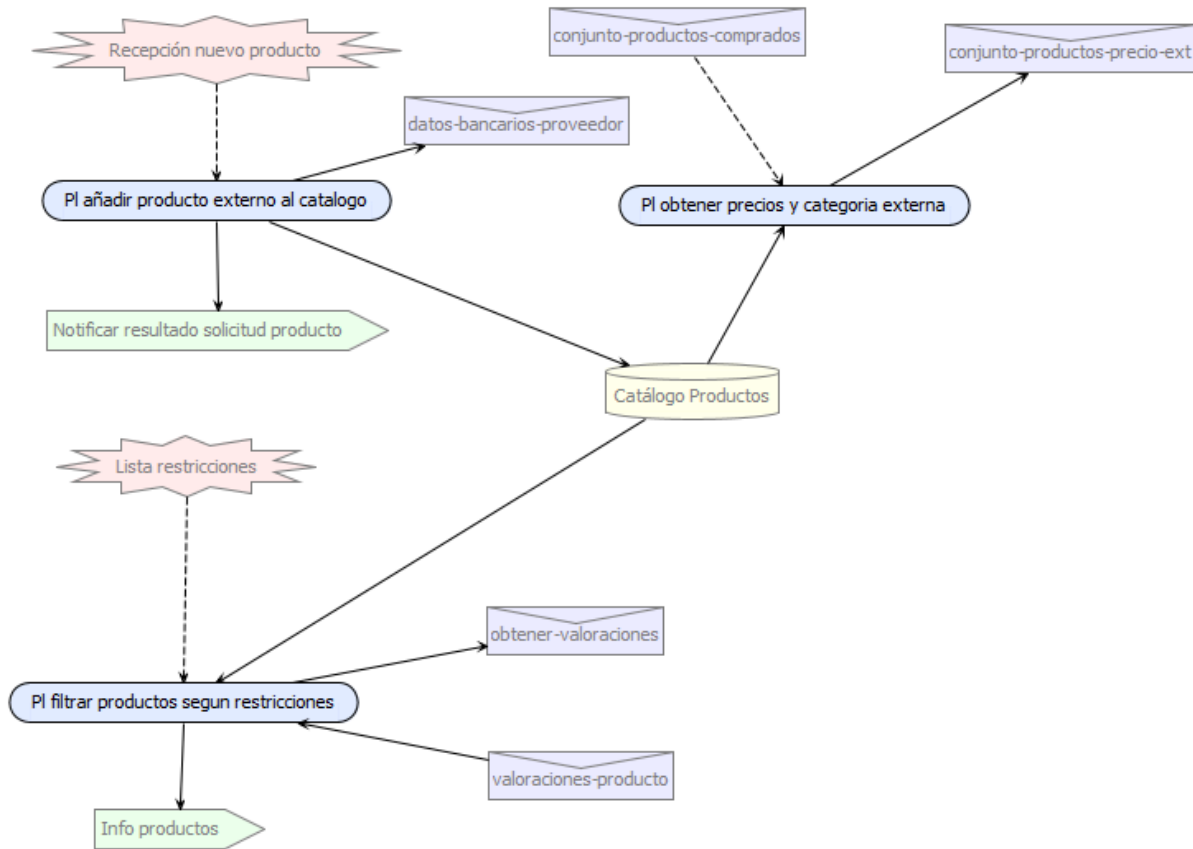
Utiliza esta información junto con los datos almacenados en Valoraciones para generar una lista de recomendaciones-productos.

Este es el núcleo de la lógica de recomendación del agente.

Plan Enviar recomendaciones: Este plan se activa una vez generadas las recomendaciones.

Se encarga de enviarlas al usuario mediante la acción Enviar sugerencia, cerrando así el proceso de recomendación.

3.2. Ag. Catalogador



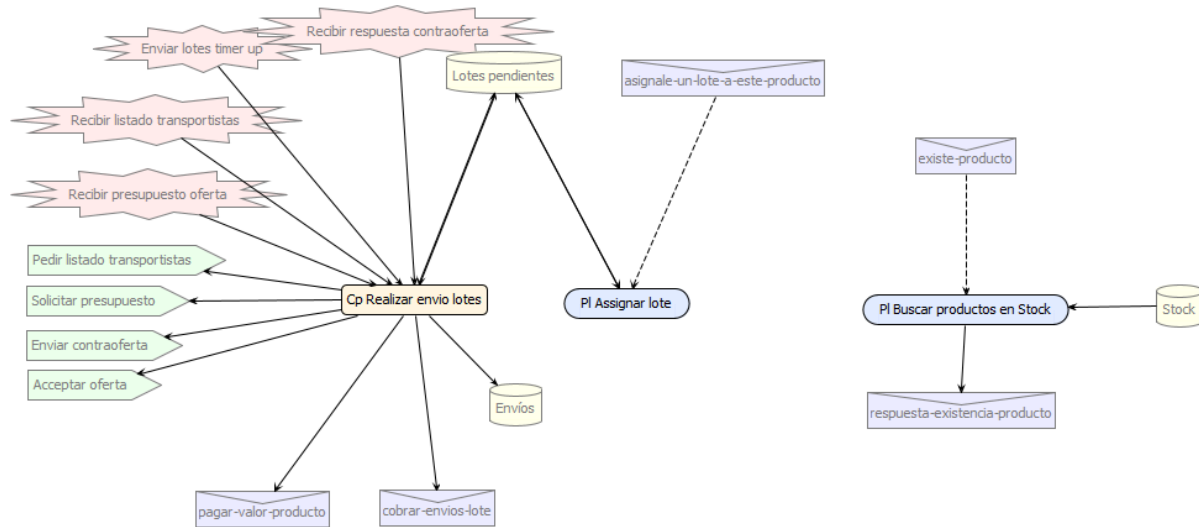
El agente Catalogador tiene tres planes simples donde solo modifica la tabla de Catálogo Productos.

PI añadir producto externo al catalogo: Este plan se activa cuando recibimos la percepción de que una tienda externa quiere añadir un producto a nuestro catálogo, el plan se encarga de comprobar que sea posible y añadir el producto a nuestro Catálogo Productos. Finaliza notificando la resolución de esta solicitud a la tienda externa.

PI filtrar productos segun restricciones: Este plan empieza cuando un usuario quiere buscar productos en nuestro catálogo así que nos envía la percepción de Lista restricciones, donde se nos dan diferentes opciones de restricciones como precio o nombre, una vez encontrados estos los productos que las cumplan en nuestro Catálogo Productos, le pide al Valorador la valoración de estos productos, si una de las restricciones es la valoración se compraban los productos otra vez para entregar una lista más detallada al usuario.

PI obtener precios y categoria externa: Este es un plan simple donde se devuelve la información de los productos solicitados en el mensaje conjunto-productos-comprados

3.3. Ag. Agente lógístico

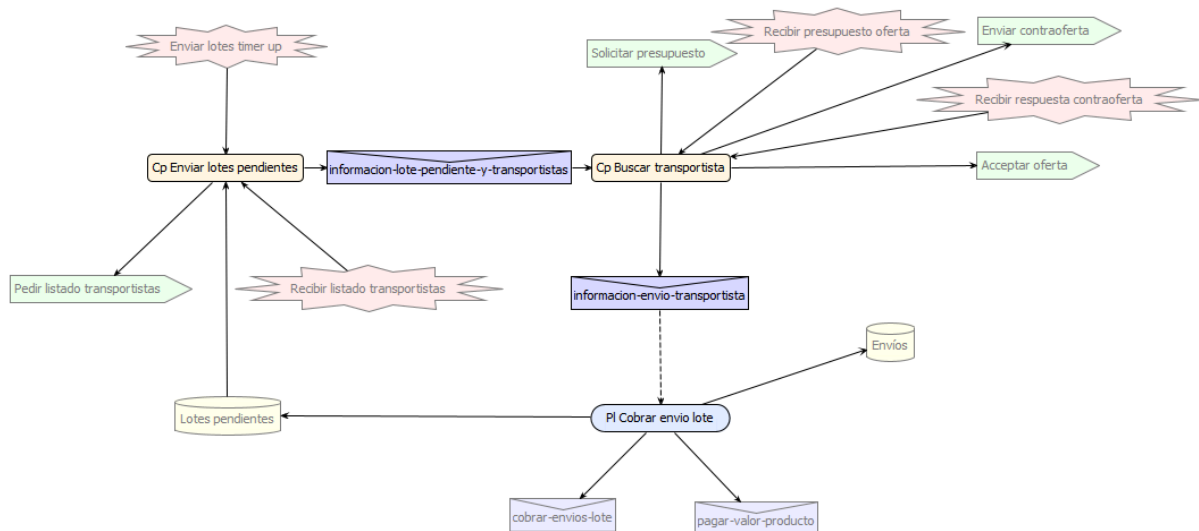


El agente Lógico se compone de dos planes sencillos y una capacidad muy grande que engloba toda la lógica del envío de un lote.

PI Buscar producto en stock: Este plan es una simple comprobación cuando el Ag. Vendedor nos da una lista de productos y el plan se encarga de marcar aquellos productos que se encuentren en el Stock del gestor logístico.

PI Asignar lote: Este plan se ejecuta cuando el Ag. Vendedor quiere asignar un producto a un lote, y envía el mensaje *asignale-un-lote-a-este-producto*, el plan se encarga de hacer lo que pide este mensaje, asignar el producto a un lote que no este lleno.

3.3.1. Cp Realizar envío lotes



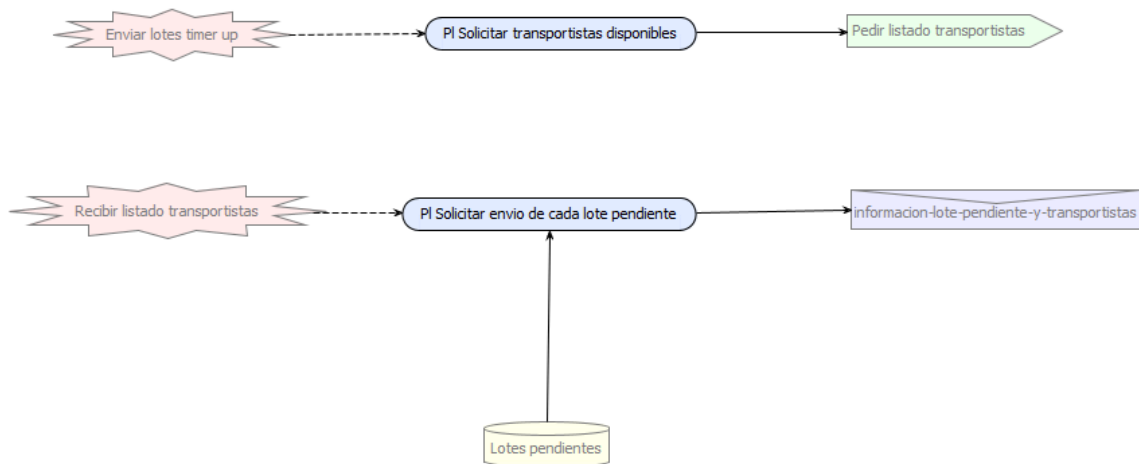
Para poder realizar esta capacidad se divide en tres trozos:

Cp Enviar lotes pendientes: Esta es una capacidad muy simple que se activa periódicamente, se encarga de solicitar el listado de transportistas disponibles y obtener el listado del lotes pendientes para iterar sobre ellos e ir solicitando los envíos.

Cp Buscar transportistas: Una vez nos llega la información de un lote y un transportista esta capacidad se encarga de realizar la lógica de negociar con un transportista, donde se intercambian acciones y percepciones hasta que decidimos aceptar la oferta, en ese momento se envía el mensaje con la informacion-envio-transportista.

PI Cobrar envio lote: Este último proceso es un plan simple que se encarga de solicitar al Tesorero las transferencias pertinentes, guardar el envío y eliminar el lote de lotes pendientes.

3.3.1.1 Cp Enviar lotes pendientes

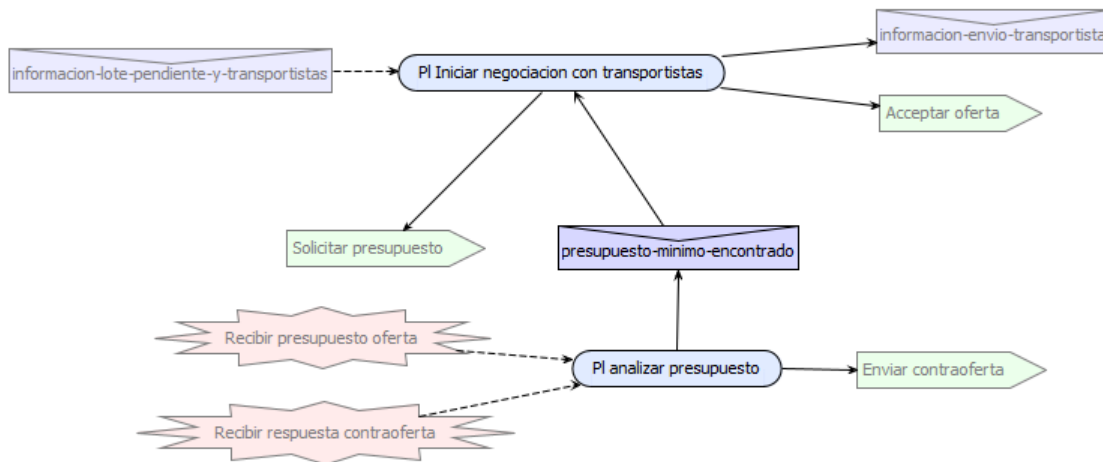


Esta capacidad se divide en dos planes muy simples:

PI Solicitar transportistas disponibles: Como el nombre ya indica este es un plan simple, pero necesario que se encarga de solicitar el listado de transportistas para poder comenzar a gestionar los envíos.

PI Solicitar envio de cada lote pendiente: Este segundo plan se encarga de iterar sobre los lotes pendientes y ordenar su envío pasando toda la informacion necesaria.

3.3.1.2 Cp Buscar transportistas

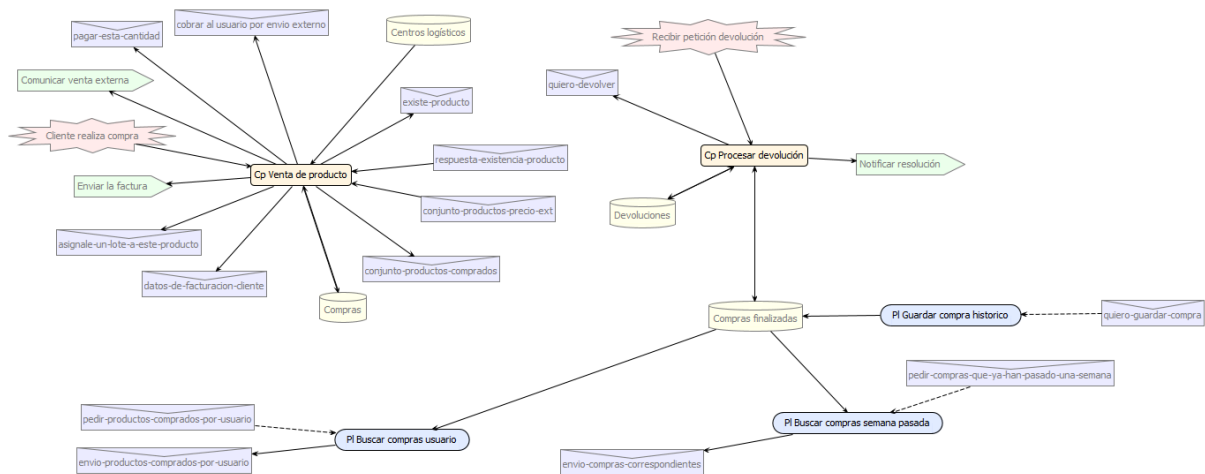


Esta capacidad tiene mas logica en los planes, ya que se encarga de gestionar la negociacion con los transportistas.:

PI Iniciar negociacion con transportistas: Este plan se encarga de solicitar a todos los transportistas una oferta inicial enviando los datos del envío solicitado, a través de la acción Solicitar presupuesto, i se espera a recibir el mensaje de presupuesto-minimo-encontrado de cada transportista, entre estos busca el más barato y le envía la acción Aceptar oferta. Finalmente envía el mensaje informacion-envio-transportista con los datos del envío completos.

PI Analizar presupuesto: Este plan es el que recibe la Percepción de presupuesto oferta de un transportista, se guarda esta oferta y envía la acción Enviar contraoferta para solicitar una rebaja del precio. Entonces recibe la Percepción de Recibir respuesta contraoferta, que puede ser de tres tipos, aceptada, donde se envía el valor aceptado en el mensaje presupuesto-minimo-encontrado, rechazado donde se envía el mismo mensaje pero con el valor guardado de la anterior oferta, y contraoferta, donde se propone un precio inferior al original, pero superior al de nuestra contraoferta, entonces se vuelva a enviar la acción Enviar contraoferta con un precio inferior al recibido pero superior a nuestra anterior contraoferta.

3.4. Ag. Ventas



El agente Ventas es el encargado de gestionar todo el ciclo comercial e interactuar directamente con las solicitudes de los usuarios. Se ocupa tanto de procesar las nuevas intenciones de compra (verificando la disponibilidad de stock), como de gestionar las peticiones de devolución, manteniendo en todo momento un registro histórico consolidado de las operaciones finalizadas.

El agente se estructura en dos capacidades principales: Cp Procesar devolución y Cp Venta de producto, además de varios planes a nivel general que facilitan las consultas sobre el historial de ventas y la emisión de facturas.

Plan Guardar compra histórico: Este plan se activa cuando el agente recibe el mensaje quiero-guardar-compra desde otro punto del sistema (por ejemplo, una vez confirmado el pago). El plan se encarga de almacenar la información de la transacción en el recurso Compras finalizadas y ejecuta la acción Enviar la factura hacia el entorno (el usuario). Su función es dar por concluida una transacción exitosa, persistir los datos de forma definitiva y emitir el comprobante correspondiente al cliente.

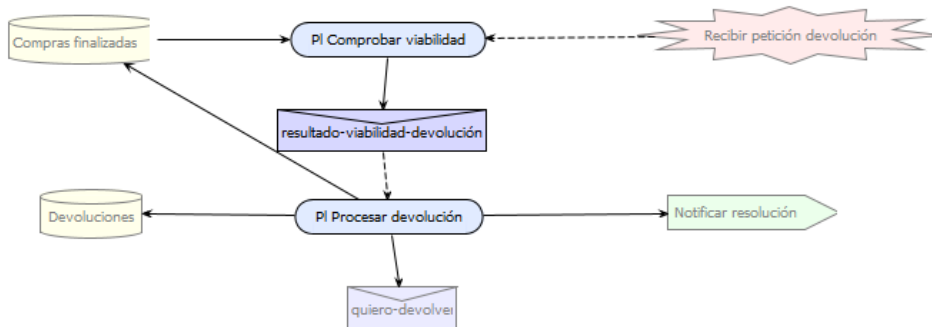
Plan Buscar compras usuario: Este plan se activa al recibir el mensaje pedir-productos-comprados-por-usuario. El agente consulta el recurso Compras finalizadas para obtener el historial específico de un cliente y devuelve la información a través del mensaje envio-productos-comprados-por-usuario. Su objetivo es proveer a otros agentes de los datos históricos necesarios sobre qué productos ha adquirido un usuario concreto.

Plan Buscar compras semana pasada: Este plan se activa cuando otro agente requiere información temporal mediante el mensaje pedir-compras-que-ya-han-pasado-una-semana.

El plan lee el recurso Compras finalizadas, filtra los registros que cumplen con el criterio temporal y responde emitiendo el mensaje envio-compras-correspondientes.

Su función es facilitar la búsqueda de compras recientes para procesos paralelos, como puede ser la solicitud de feedback y valoraciones.

3.4.1 Capacidad Procesar devolución



Esta capacidad agrupa la lógica encargada de recibir, evaluar y ejecutar las peticiones de devolución realizadas por los usuarios, asegurando que se cumplan las normativas de la tienda.

Plan Comprobar viabilidad: Este plan se activa cuando el agente recibe la percepción del entorno Recibir petición devolución.

Se encarga de consultar el recurso Compras finalizadas para verificar si la compra original existe y si cumple los requisitos (como plazos o estado) para ser devuelta. El resultado se encapsula y se envía a través del mensaje interno resultado-viabilidad-devolución.

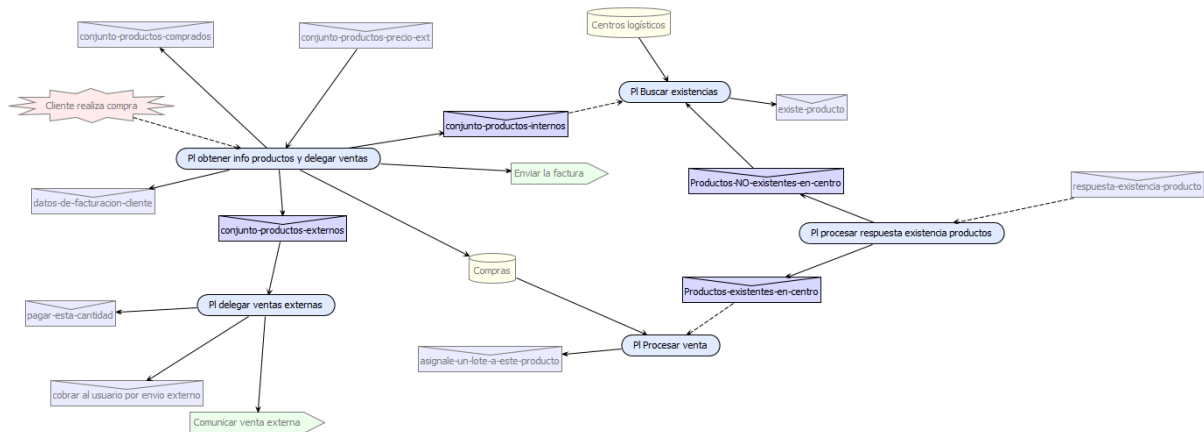
Su objetivo es actuar como filtro inicial, validando la legitimidad de la solicitud de devolución antes de iniciar cualquier trámite logístico o económico.

Plan Procesar devolución: Este plan se activa al recibir el mensaje interno resultado-viabilidad-devolución.

Si la devolución es viable, el plan actualiza los recursos de creencias Devoluciones y altera el estado en Compras finalizadas. Seguidamente, envía el mensaje quiero-devolver (para que el tesorero inicie el reembolso) y ejecuta la acción Notificar resolución para informar al usuario.

Su función es hacer efectiva la devolución en el sistema, desencadenar las operaciones monetarias correspondientes y cerrar la comunicación con el cliente.

3.4.2 Venta de producto



Esta capacidad concentra la compleja lógica necesaria para procesar una nueva solicitud de compra, delegar el envío o rastrear el producto a través de los centros logísticos y, en caso de éxito, materializar la orden de venta.

Plan obtener info productos y delegar ventas: Este es el Plan inicial que se activa al realizar una compra, este recibe la percepción de Cliente realiza compra y guarda la compra en la tabla de Compras, a continuación cumple 4 tareas fundamentales; genera la acción Enviar la Factura, para notificar al cliente que se compra se ha registrado, envía el mensaje datos-de-facturacion-cliente al financiero para evitar arrastrar estos datos innecesariamente, solicita al Catalogador los datos de los productos comprados y finalmente delega los conjuntos de productos externos e internos a los planes pertinentes para su gestión.

Plan delegar ventas externas: El plan para cada proveedor genera la acción de Comunicar venta externa con el fin de solicitar a la empresa externa el envío de un producto externo, posteriormente envía dos mensajes al tesorero, uno para pagar a la tienda externa y otro para cobrar al usuario el valor de los productos.

Plan Buscar existencias: Este plan se activa de dos maneras: ante el mensaje propio de conjunto-productos-internos, o a través del mensaje interno de iteración Productos-NO-existent-en-centro.

El plan consulta el recurso Centros logísticos para identificar a qué centro debe preguntar por el stock y envía el mensaje externo existe-producto hacia el agente correspondiente. Su función es iniciar y, si es necesario, mantener el rastreo iterativo del inventario físico a través de la red logística.

Plan Procesar respuesta existencia productos: Este plan se activa cuando el sistema devuelve el mensaje respuesta-existencia-producto.

Analiza la respuesta recibida y bifurca el flujo mediante mensajería interna: si no hay stock en ese centro, emite el mensaje Productos-NO-existent-s-en-centro (lo que vuelve a disparar el plan anterior para seguir buscando). Si sí hay stock, emite el mensaje Productos-existent-s-en-centro.

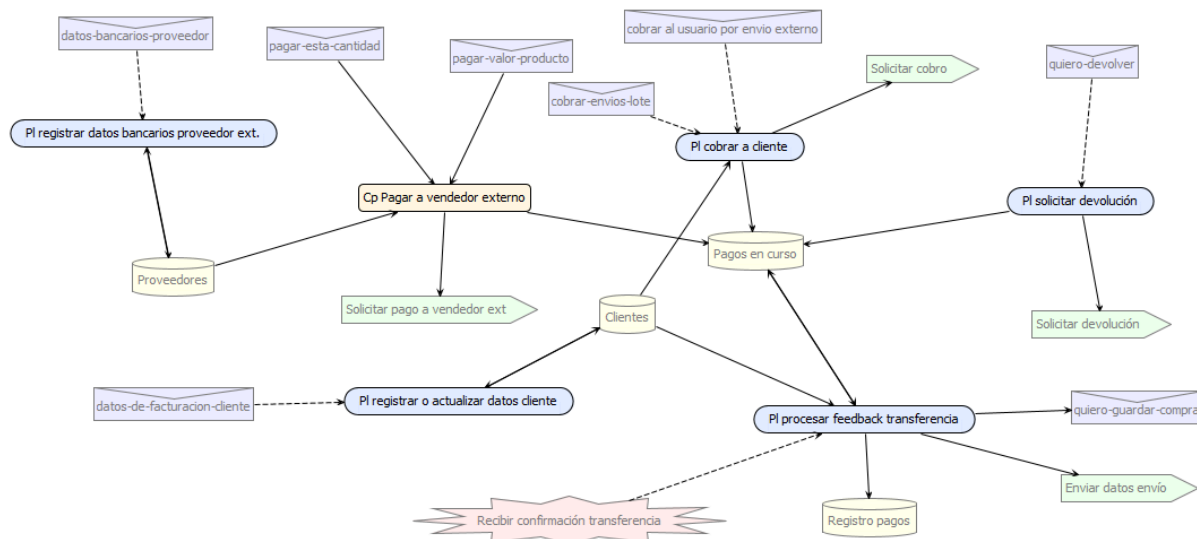
Su objetivo es actuar como un evaluador lógico que decide si la búsqueda debe continuar en otro almacén o si ya se puede proceder con la venta.

Plan Procesar venta: Este plan se activa de forma definitiva cuando recibe el mensaje interno Productos-existent-s-en-centro.

Al tener la confirmación de stock, el plan registra el pedido en el recurso Compras y desencadena la operativa para recuperar los datos de la compra con el fin de enviar el mensaje asigne-un-lote-a-este-producto (para iniciar el empaquetado y envío).

Su función es consolidar la creación formal de la venta, registrando la orden y delegando la ejecución material a las áreas logísticas.

3.5. Ag. Tesorero



El agente Tesorero es el encargado de gestionar todo el ciclo financiero y de transacciones monetarias dentro del sistema. Se ocupa tanto de realizar cobros a los clientes y gestionar devoluciones, como de coordinar los pagos a los vendedores externos, manteniendo en todo momento un registro actualizado del estado de las operaciones.

El agente se estructura en una capacidad principal: Cp Pagar a vendedor externo, además de varios planes a nivel general que coordinan el comportamiento global de cobros, devoluciones y confirmaciones de pago.

Plan Cobrar a cliente: Este plan se activa cuando el agente recibe los mensajes quiero cobrar al usuario o cobrar-envios-lote por parte de otros agentes del sistema.

El plan se encarga de registrar esta nueva obligación financiera en el recurso de creencias Pagos en curso y ejecuta la acción Solicitar cobro hacia el entorno (por ejemplo, una pasarela de pago).

Su función es centralizar e iniciar las peticiones de cobro hacia los usuarios, manteniendo el seguimiento de la transacción.

Plan Solicitar devolución: Este plan se activa cuando el agente recibe el mensaje quiero-devolver.

El agente consulta o actualiza el recurso Pagos en curso y emite la orden hacia el entorno mediante la acción Solicitar devolucion.

Su objetivo es procesar correctamente los reembolsos, interactuando con el entorno financiero y asegurando que el estado del dinero quede debidamente trackeado.

Plan Procesar feedback transferencia: Este plan se activa con la percepción del entorno Recibir confirmacion transferencia, que indica que un movimiento de dinero se ha completado físicamente.

El plan consulta el recurso Pagos en curso para validar la operación, almacena la información definitiva en el recurso Registro pagos para su persistencia histórica, y notifica al sistema enviando el mensaje quiero-guardar-compra.

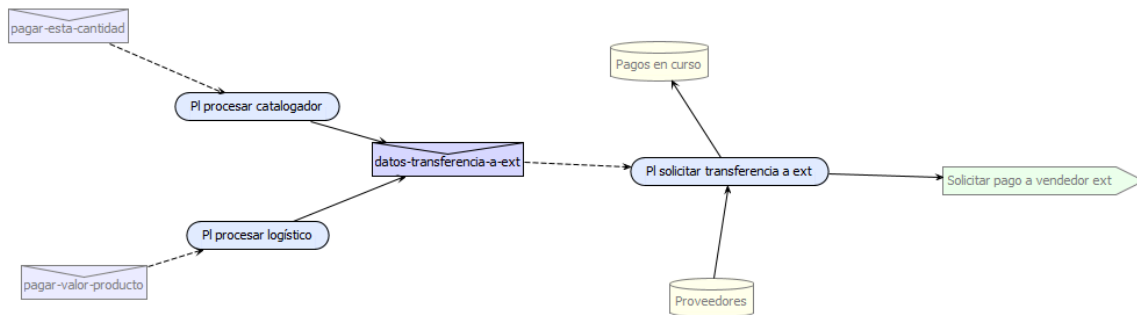
Si fuese necesario consulta el recurso Clientes para notificarle la resolución del envío.

Su función es cerrar el ciclo de las transacciones financieras, confirmando que el dinero se ha movido y permitiendo que otros agentes puedan consolidar la compra.

Plan registrar datos bancarios proveedor ext: Este simple plan se activa con el mensaje datos-bancarios-proveedor donde se consulta y actualiza el recurso de Proveedores con el fin de mantener actualizada la información de “contacto” de los proveedores.

Plan registrar o actualizar datos cliente: Este simple plan se activa con el mensaje datos-de-facturacion-cliente para consulta y actualiza el recurso de Clientes con el fin de mantener actualizada la información de facturación de los clientes.

3.5.1. Capacidad Pagar a vendedor externo



Esta capacidad agrupa la lógica necesaria para recibir distintas solicitudes de pago hacia proveedores o vendedores externos (ya provengan del ámbito logístico o de catálogo), unificarlas y ejecutar la orden de transferencia al exterior.

Plan Procesar catalogador: Este plan se activa cuando se recibe el mensaje de entrada pagar-esta-cantidad.

Se encarga de procesar los datos económicos provenientes de la lógica de catálogo y los transforma enviando el mensaje interno datos-transferencia-a-ext.

Su objetivo es estandarizar la petición de pago para que pueda ser procesada de forma unificada.

Plan Procesar logístico: Este plan se activa al recibir el mensaje de entrada pagar-valor-producto.

De forma similar al plan anterior, procesa la información de pago proveniente de la rama logística y emite también el mensaje interno datos-transferencia-a-ext.

Su función es extraer y preparar los datos de los envíos o valor del producto para integrarlos en el flujo de pagos externos.

Plan Solicitar transferencia a ext: Este plan se activa cuando se recibe el mensaje interno datos-transferencia-a-ext (generado por cualquiera de los dos planes anteriores).

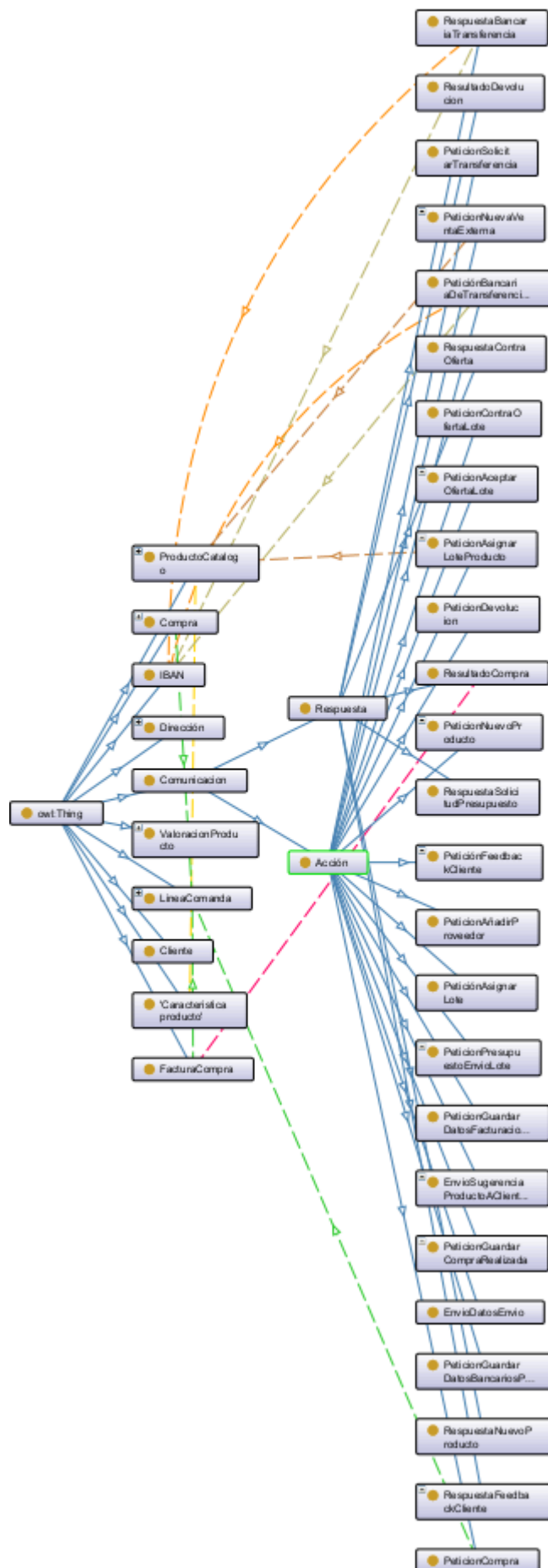
A partir de esta información unificada, el plan registra el intento de pago en el recurso Pagos en curso y juntamente con los datos obtenidos del registro Proveedores ejecuta la acción hacia el entorno Solicitar pago a vendedor ext.

Su función es actuar como embudo final de la capacidad: toma los datos consolidados, deja constancia de que la operación se ha iniciado y lanza la instrucción definitiva a la pasarela externa.

4. Ontología

La ontología del sistema define los conceptos compartidos entre los agentes, así como sus atributos y relaciones. Esta ontología permite que los distintos agentes intercambien información mediante mensajes siguiendo un vocabulario común y consistente.

En este sistema, los conceptos de comunicación corresponden principalmente a los mensajes definidos en los protocolos de interacción de Prometheus.



4.1. Conceptos

La ontología del sistema define los elementos principales del dominio de la aplicación y los conceptos utilizados en la comunicación entre agentes. En la jerarquía se distinguen conceptos de negocio, como productos, compras o clientes, y conceptos de comunicación, que representan los mensajes intercambiados entre agentes.

4.1.1. Conceptos principales del dominio

4.1.1.1. ProductoCatalogo

Representa un producto disponible en el sistema. Es uno de los elementos centrales de la aplicación, ya que puede ser comprado, catalogado, valorado y recomendado.

4.1.1.2. Característica producto

Representa una de las características asociadas a un producto, como información descriptiva o propiedades relevantes para su clasificación y búsqueda.

4.1.1.3. Cliente

Representa al usuario comprador del sistema. Puede realizar compras, recibir recomendaciones y generar valoraciones sobre productos.

4.1.1.4. Compra

Representa una operación de compra realizada por un cliente. Se relaciona con líneas de comanda, productos adquiridos y facturas.

4.1.1.5. LíneaComanda

Representa una línea concreta dentro de una compra o comanda. Permite detallar qué producto se ha comprado y en qué cantidad.

4.1.1.7. FacturaCompra

Representa la factura generada como resultado de una compra.

4.1.1.8. ValoracionProducto

Representa la valoración realizada sobre un producto. Puede incluir información como la puntuación o el resultado de una valoración.

4.1.1.9. IBAN

Clase que representa el IBAN de una entidad, o el del propio sistema, con el fin de poder cobrarles o pagarles

4.1.1.10. Dirección

Clase que representa una dirección de entrega, o de un centro logístico para poder informar al transportista.

4.1.2. Conceptos de comunicación

4.1.2.1. Comunicacion

Concepto general que agrupa los mensajes intercambiados entre agentes en los que el sistema se ve modificado. Se divide en dos grandes tipos: acciones o peticiones enviadas y respuestas recibidas.

4.1.2.2. Acción

Agrupa los mensajes que representan peticiones o acciones enviadas por un agente a otro.

Subconceptos:

- **EnvioDatosEnvio:** Clase que representa la acción mandada al cliente por parte del agente tesorero que contiene los datos del envío.
- **PeticionAceptarOfertaLote:** Clase que representa la petición enviada por el agente logístico a un transportista para aceptar la oferta de un envío de lote..
- **PeticionAsignarLoteProducto:** Clase que representa la petición mandada por el agente Ventas hacia un centro logístico para asignar una cantidad de un producto comprado en los lotes pendientes de envío.
- **PeticionAñadirProveedor:** Clase que representa la petición mandada por el agente Catalogo al agente Tesorero para añadir o actualizar los datos bancarios de un proveedor.
- **PeticionCompra:** Clase que representa la petición de compra mandada por el agente asistente del usuario al agente vendedor - Percepción de Ventas (Cliente realiza compra).
- **PeticionContraOfertaLote:** Clase que representa la contraoferta enviada por el centro logístico a un transportista solicitando un precio más bajo para el envío.

- **PeticionGuardarCompra:** Clase que representa la petición por parte del tesorero al vendedor para guardar la compra realizada.
- **PeticionGuardarCompraRealizada:** Clase que representa el mensaje enviado por parte del tesorero al agente de ventas comunicándole que una compra ha sido efectuada y que hay que guardar la compra como realizada.
- **PeticionGuardarDatosBancariosProveedor:** Clase que representa la petición mandada por el catalogador para registrar los datos bancarios de un proveedor externo.
- **PeticionGuardarDatosFacturacionClientes:** Clase que representa la petición mandada por el agente Ventas al Tesorero para guardar los datos de facturación de un Cliente.
- **PeticionNuevaVentaExterna:** Clase que representa la petición mandada por el agente Ventas al actor Empresa Venedora para delegar el envío de uno de sus productos.
- **PeticionNuevoProducto:** Clase que representa la petición mandada por el actor Empresa Venedora a Catalogo para que añada un producto (datos de producto y datos del proveedor).
- **PeticionPresupuestoEnvioLote:** Clase que representa la petición enviada por el agente logístico a un transportista para obtener el presupuesto de el envío de un lote, dadas sus características.
- **PeticionSolicitarTransferencia:** Clase que representa la solicitud de una transferencia entre dos entidades, enviada por Ventas y Agente Logístico para realizar cobros y pagos. Contiene un campo que indica si es una devolución al cliente, un pago a vendedor externo o un cobro al cliente.
- **PeticiónAsignarLote:** Clase que representa el mensaje "Asigne-un-lote-a-este-producto" del agente ventas hacia Agente logístico.
- **PeticiónBancariaDeTransferencia:** Clase que representa la acción mandada por parte del tesorero a la entidad bancaria solicitando la transferencia entre dos entidades.
- **PeticiónDelegarVentaExterna:** Clase que representa la acción "Comunicar venta externa", mandada por el agente de ventas al actor externo que representa una empresa externa.
- **PeticiónDevolverDineroACliente:** Clase que representa el mensaje enviado por parte del agente de ventas al tesorero para solicitar la devolución del dinero de un producto que el cliente quiere devolver.
- **PeticiónFeedbackCliente:** Clase que representa la acción mandada al cliente para pedirle valoración sobre un producto ya comprado.

- **EnvioSugerenciaProductoACliente:** Clase que representa el mensaje enviado por parte del valorador al cliente conteniendo productos que pueden ser de su interés.

4.1.2.3. Respuesta

Agrupar los mensajes que representan respuestas enviadas como resultado de una petición previa.

Subconceptos:

- **RespuestaBancariaTransferencia:** Clase que representa la confirmación enviada por parte de la entidad bancaria al tesorero de una transferencia solicitada anteriormente.
- **RespuestaContraOferta:** Clase que representa la respuesta a la contraoferta por parte de un transportista al agente logístico.
- **RespuestaFeedbackCliente:** Clase que representa el feedback enviado por parte del cliente al agente valorador puntuando un producto que ha comprado (comentario opcional).
- **RespuestaNuevoProducto:** Clase que representa la respuesta mandada por el agente Ventas al Actor Empresa Venedora que informa de la resolución de su solicitud.
- **RespuestaSolicitudPresupuesto:** Clase que representa el presupuesto para el envío de un lote enviado por un transportista al agente logístico.
- **ResultadoCompra:** Clase que representa la respuesta (Factura) del agente Ventas para el Actor cliente - (Acción Enviar la factura).

4.2. Atributos

Los atributos de la ontología se definen como Data Properties. Estas propiedades permiten asociar valores literales a los conceptos del sistema, como identificadores, nombres, precios, cantidades, direcciones o resultados booleanos.

En cada atributo se indica su dominio, que representa la clase o concepto sobre el que se aplica, y su rango, que indica el tipo de dato que puede tomar el atributo.

4.2.1. cantidadProducto

Representa la cantidad de unidades de un producto dentro de una línea de compra o compra.

Dominio: LineaComanda

Rango: xsd:int

4.2.2. cantidadTransferencia

Representa la cantidad da valor monetario en una transferencia.

Dominio: PeticiónBancariaDeTransferencia / PeticionSolicitarTransferencia / RespuestaBancariaTransferencia

Rango: xsd:float

4.2.3. característicasProducto

Representa las características descriptivas asociadas a un producto, como propiedades técnicas, descripción general o información relevante para su clasificación.

Dominio: Caracteristica productos

Rango: xsd:string

4.2.4. direccion

Representa la dirección asociada a un cliente o a una operación de compra/facturación.

Dominio: Cliente / EnvioDatosEnvio

Rango: xsd:string

4.2.5. fechaEntrega

Representa la fecha de entrega de un producto.

Dominio: EnvioDatosEnvio / PeticionAsignarLoteProducto / PeticionCompra / PeticionPresupuestoEnvioLote / RespuestaSolicitudPresupuesto / RespuestaContraOferta

Rango: xsd:dateTime

4.2.6. idCliente

Identificador único de un cliente dentro del sistema.

Permite relacionar al cliente con compras, facturas, productos comprados o peticiones realizadas por otros agentes.

Dominio: Cliente / PeticionGuardarDatosFacturacionClientes /
PeticionGuardarCompraRealizada

Rango: xsd:string

4.2.7. idCompra

Identificador único de una compra.

Permite referenciar una operación de compra concreta en líneas de comanda, facturas, peticiones o respuestas de compra.

Dominio: Compra

Rango: xsd:string

4.2.8. idProducto

Identificador único de un producto.

Se utiliza para referenciar productos en el catálogo, en compras, en valoraciones y en mensajes de comunicación entre agentes.

Dominio: ValoracionProducto / LineaComanda / ProductoCatalogo

Rango: xsd:string

4.2.9. mensajePersonalizado

Representa un texto largo no definido que se envía o recibe hacia otras entidades, como por ejemplo el feedback de un producto.

Dominio: EnvioSugerenciaProductoACliente / RespuestaFeedbackCliente /
RespuestaNuevoProducto / PeticiónFeedbackCliente

Rango: xsd:string

4.2.10. nombreMarca

Representa el nombre de la marca asociada a un producto.

Este atributo permite identificar la marca del producto y puede ser utilizado en búsquedas, filtros o recomendaciones.

Dominio: Caracteristica producto

Rango: xsd:string

4.2.11. nombreProducto

Representa el nombre del producto.

Permite identificar de forma legible un producto dentro del catálogo, en una compra o en una respuesta entre agentes.

Dominio: Producto

Rango: xsd:string

4.2.12. nombreProveedor

Representa el nombre de un proveedor.

Permite identificar de forma legible uno de los proveedores de productos externos.

Dominio: PeticionNuevoProducto / PeticionGuardarDatosBancariosProveedor

Rango: xsd:string

4.2.13. nombreTransportista

Representa el nombre de un transportista.

Permite identificar de forma legible un transportista.

Dominio: EnvioDatosEnvio

Rango: xsd:string

4.2.14. numeroIBAN

Representa el numero IBAN de cualquier entidad que el sistema este interesado en guardarlo (El propio sistema, proveedores, transportistas, clientes).

Permite identificar el contacto bancario de cualquier entidad.

Dominio: EnvioDatosEnvio / PeticionGuardarDatosFacturacionClientes / IBAN / PeticionGuardarDatosBancariosProveedor

Rango: xsd:string

4.2.15. participanteTransferencia

Representa un de los dos participantes de una transferencia (origen o destinatario).

Dominio: PeticionSolicitarTransferencia

Rango: xsd:string

4.2.16. pesoLote

Representa un de los dos participantes de una transferencia (origen o destinatario).

Dominio: PeticionPresupuestoEnvioLote / PeticionAceptarOfertaLote

Rango: xsd:float

4.2.17. precioLineaComanda

Representa el precio asociado a una línea de comanda.

Normalmente corresponde al coste de una cantidad determinada de un producto dentro de una compra.

Dominio: LineaComanda

Rango: xsd:float

4.2.18. precioOfertaTransporte

Representa el precio asociado a un envío por parte de un transportista.

Normalmente corresponde al coste de una cantidad determinada de un producto dentro de una compra.

Dominio: RespuestaSolicitudPresupuesto / RespuestaContraOferta /
PetitionContraOfertaLote

Rango: xsd:float

4.2.19. precioProducto

Representa el precio de un producto.

Se utiliza en el catálogo, en procesos de compra y en el cálculo del precio de líneas de comanda o facturas.

Dominio: ProductoCatalogo

Rango: xsd:float

4.2.20. precioProductoMaximo

Representa el precio máximo utilizado como criterio de búsqueda o filtrado de productos.

Este atributo permite limitar los resultados de una búsqueda a productos cuyo precio no supere un valor determinado.

Dominio: Caracteristica producto

Rango: xsd:float

4.2.21. precioProductoMinimo

Representa el precio mínimo utilizado como criterio de búsqueda o filtrado de productos.

Este atributo permite limitar los resultados de una búsqueda a productos cuyo precio sea igual o superior a un valor determinado.

Dominio: Caracteristica producto

Rango: xsd:float

4.2.22. precioTotalFactura

Representa el importe total de una factura de compra.

Este atributo resume el coste final de la compra facturada.

Dominio: FacturaCompra

Rango: xsd:float / xsd:decimal

4.2.23. tipoTransferencia

Representa el tipo de transferencia solicitado por parte de un agente al financiero (devolucion, pago, cobro)

Dominio: PeticionSolicitarTransferencia

Rango: xsd:string

4.2.24. valoracionProducto

Representa la valoración asignada a un producto.

Se utiliza para registrar o consultar la opinión de los clientes sobre un producto determinado.

Dominio: ValoracionProducto

Rango: xsd:float

4.2.24. éxitoNuevoProducto

Respuesta por parte del sistema a una empresa confirmando el éxito de su solicitud.

Dominio: RespuestaNuevoProducto

Rango: xsd:boolean

4.2.25. éxitoTransferencia

Respuesta por parte de la entidad bancaria al sistema confirmando el éxito de su solicitud.

Dominio: RespuestaBancariaTransferencia

Rango: xsd:boolean

4.3. Relaciones

Las relaciones de la ontología se definen como **Object Properties**. Estas propiedades permiten conectar conceptos entre sí, representando asociaciones entre entidades del dominio o entre mensajes de comunicación y los datos que transportan.

A diferencia de las *Data Properties*, que conectan una clase con un valor literal, las *Object Properties* conectan una clase con otra clase de la ontología.

4.3.1. IBAN_destino

Relaciona un concepto con un IBAN, marcado como destinatario.

Sirve para poder relacionar un concepto con un IBAN pero asegurándonos de que este sea el del destinatario.

Dominio: PeticiónBancariaDeTransferencia, RespuestaBancariaTransferencia.

Rango: IBAN

4.3.2. IBAN_origen

Relaciona un concepto con un IBAN, marcado como origen.

Sirve para poder relacionar un concepto con un IBAN pero asegurándonos de que este sea el de origen.

Dominio: PeticiónBancariaDeTransferencia, RespuestaBancariaTransferencia.

Rango: IBAN

4.3.3. contiene_caracteristicas_producto

Relaciona un producto con sus características asociadas.

Esta relación permite indicar que un producto puede tener una o varias características descriptivas, como marca, propiedades técnicas u otros detalles del producto.

Dominio: ProductoCatálogo

Rango: Caracteristica productos

4.3.4. contiene_factura

Relaciona una compra o resultado de compra con la factura generada.

Esta relación permite asociar una operación de compra con su correspondiente FacturaCompra.

Dominio: ResultadoCompra

Rango: FacturaCompra

4.3.5. contiene_linea

Relaciona un concepto con una línea de comanda concreta.

Permite representar qué línea de comanda está siendo comprobada o referenciada.

Dominio: PeticionAsignarLoteProducto, PeticionNuevaVentaExterna

Rango: LineaComanda

4.3.6. contiene_lineas

Relaciona un concepto con las líneas de comanda que la forman.

Una compra puede estar compuesta por una o varias líneas de comanda, donde cada línea representa un producto comprado y su cantidad.

Dominio: Compra

Rango: LineaComanda

4.3.7. contiene_producto

Relaciona un concepto con un producto.

Esta relación permite representar el producto del qual se pide feedback.

Dominio: PeticiónFeedbackCliente

Rango: ProductoCatálogo

4.3.8. contiene_productos

Relaciona un concepto con los productos asociados.

Esta relación permite representar qué productos forman parte de una compra o qué productos son devueltos en una respuesta de productos comprados por usuario.

Dominio: ResultadoProductosCompradosUsuario

Rango: ProductoCatálogo

4.3.9. contiene_valoración

Relaciona un concepto con una valoración a un producto

Esta relación permite representar que una respuesta sobre valoraciones de producto puede incluir una instancia de ValoracionProducto.

Dominio: EnvioSugerenciaProductoACliente, RespuestaFeedbackCliente

Rango: ValoracionProducto

4.3.10. compra_realizada

Relaciona una petición de guardar una compra con su compra correspondiente.

Sirve para poder referenciar la compra la cual se quiere registrar como ya ejecutada.

Dominio: PeticionGuardarCompraRealizada

Rango: Compra

4.3.11. dirección_destino

Relaciona un concepto con una dirección, marcada como destino.

Sirve para poder relacionar un concepto con una dirección pero asegurándonos de que esta sea la de destino.

Dominio: PeticionPresupuestoEnvioLote, PeticionAceptarOfertaLote,
PeticionAsignarLoteProducto, PeticionNuevaVentaExterna

Rango: Dirección

4.3.12. dirección_origen

Relaciona un concepto con una dirección, marcada como origen.

Sirve para poder relacionar un concepto con una dirección pero asegurándonos de que esta sea la de origen.

Dominio: PeticionPresupuestoEnvioLote, PeticionAceptarOfertaLote.

Rango: Dirección

5. Explicación y justificación de la implementación en python

La implementación en Python se ha construido siguiendo la estructura definida en el diseño con Prometheus Design Tool y la ontología OWL. El objetivo principal no era crear una aplicación monolítica de comercio electrónico, sino un sistema multiagente donde cada responsabilidad estuviera encapsulada en un proceso independiente, con comunicación explícita mediante mensajes RDF/FIPA y con un directorio de servicios que permitiera descubrir agentes dinámicamente.

5.1 Arquitectura general de la implementación

Cada agente se ha implementado como un servicio Flask independiente, con un endpoint común /message para recibir comunicaciones RDF/FIPA. Esta decisión permite ejecutar los agentes en procesos distintos e incluso, si se quisiera desplegar fuera del entorno de demostración, en máquinas distintas. En la práctica local, todos los servicios se levantan con develop.sh, que inicia el Directorio de Servicios, el Logger, el Cliente, Catalogador, Ventas, Valorador, Tesorero, Entidad Bancaria, varias Empresas Vendedoras externas, varios Transportistas y varias instancias de Centro Logístico.

La pieza central de comunicación es AgentCommunication.py. Este módulo encapsula la creación y lectura de los mensajes ACL, la serialización RDF, los identificadores de conversación, las performativas FIPA y los conceptos definidos en la ontología. Así, los agentes no intercambian diccionarios Python arbitrarios, sino grafos RDF con contenido tipado como PeticionCompra, ResultadoCompra, PeticionAsignarLoteProducto, PeticionPresupuestoEnvioLote, PeticionSolicitarTransferencia, PeticionDevolucion, ResultadoDevolucion, etc. Esta separación reduce la duplicación de código y mantiene alineada la implementación con la ontología descrita en el apartado 4.

El Directorio de Servicios (DirectoryService.py) funciona como una implementación del DF. Cada agente se registra al arrancar indicando su identificador, tipo y dirección, y después los demás agentes pueden consultarlo por tipo. Para los casos donde se necesita una única instancia, por ejemplo localizar a Ventas o Catalogador, se usa una búsqueda simple. Para los casos con múltiples agentes posibles, como Centros Logísticos, Transportistas o Empresas Vendedoras, se usa una búsqueda múltiple. Esta decisión permite representar correctamente que algunos roles pueden tener varias instancias activas y que el sistema no depende de direcciones hardcodeadas.

5.2 Interfaz de cliente y experiencia de demostración

El agente Client.py ofrece una interfaz web donde el usuario puede buscar productos por restricciones, confirmar una propuesta de compra, introducir dirección de entrega, indicar IBAN y especificar un plazo máximo de entrega. La interfaz no genera localmente la factura: al confirmar la compra, el Cliente envía una *PeticionCompra* a Ventas y espera que la factura llegue posteriormente como un mensaje RDF *ResultadoCompra*. Cuando recibe este mensaje, lo almacena como notificación y permite imprimirla o guardarla en PDF desde una vista específica.

También se ha añadido en la interfaz un formulario permanente de devolución. Este formulario permite seleccionar una factura conocida, escoger si se devuelve el pedido completo o un producto concreto, indicar el motivo y añadir un comentario. Si el usuario introduce manualmente un pedido inexistente o no viable, no se bloquea en la interfaz, ya que la decisión corresponde al agente Ventas. De esta forma se respeta la separación de responsabilidades: el Cliente solo emite la intención de devolución y Ventas aplica la lógica de viabilidad.

Además, el Cliente recibe notificaciones posteriores de datos de envío, recomendaciones y solicitudes de feedback. Esto reproduce la idea de que el usuario no recibe toda la información en el instante de compra, sino que el sistema le comunica eventos posteriores cuando los agentes internos completan sus tareas.

5.3 Implementación de los agentes internos

El agente Catalogador.py centraliza el catálogo de productos y permite buscar por restricciones como nombre, marca, vendedor, rango de precio, valoración mínima y características. También registra productos enviados por empresas vendedoras externas. En el catálogo se distingue entre productos internos, productos externos gestionados por nuestros centros logísticos y productos totalmente externos. Esta distinción es importante porque afecta al circuito de envío y al circuito de pagos.

El agente Ventas.py coordina el ciclo comercial. Al recibir una compra, consulta al Catalogador para obtener información extendida de los productos, registra los datos de facturación del cliente en el Tesorero y separa las líneas según si deben pasar por nuestros centros logísticos o si son totalmente externas. Para los productos gestionados por almacén, Ventas ordena los centros logísticos por proximidad a la dirección del cliente y consulta la disponibilidad empezando por el centro más cercano. Si un centro confirma que

tiene el producto, no se consulta al resto para ese producto, evitando tráfico innecesario. Después se asigna la línea a un lote del centro correspondiente.

Para los productos totalmente externos, Ventas comunica la venta a una Empresa Vendedora externa y delega en ella el envío. En ese caso, Ventas también coordina el cobro al cliente y el pago al proveedor externo a través del Tesorero. Esto permite cubrir los tres casos documentados: producto interno, producto externo almacenado por nosotros y producto externo totalmente delegado.

El agente CentroLogistico.py representa una instancia concreta de almacén. Cada centro mantiene sus propios lotes pendientes, su dirección, una distribución inicial de stock de demostración y la información de envíos que ha gestionado. La disponibilidad de stock se consulta mediante query-if, tal como estaba previsto en el protocolo de existencia. Para facilitar la repetición de casos durante la demo, la comprobación de stock no descuenta físicamente unidades, pero conserva el comportamiento de consulta, asignación de lote y envío.

El agente Tesorero.py centraliza las operaciones financieras. Guarda datos de clientes, proveedores, pagos en curso y pagos procesados. Cuando recibe peticiones de cobro, pago a proveedor o devolución, construye la solicitud correspondiente para EntidadBancaria.py. La Entidad Bancaria actúa como actor externo configurable y responde con confirmaciones de transferencia. Una vez confirmada una transferencia de envío, el Tesorero es quien envía al Cliente los datos finales de envío, evitando que el Centro Logístico comunique directamente al usuario información que depende de la confirmación bancaria.

El agente Valorador.py implementa las capacidades de pedir feedback y generar recomendaciones. El feedback solo se solicita cuando existe una compra finalizada con fecha de entrega registrada y ha transcurrido el margen configurado desde dicha entrega. Para las recomendaciones se usa una puntuación bayesiana que combina valoraciones previas, afinidad con el historial del cliente, etiquetas de producto y popularidad. Con ello se evita recomendar productos vacíos o completamente arbitrarios, y se justifica que las sugerencias se basen en información histórica del sistema.

5.4 Implementación de actores externos

Aunque algunos actores externos no forman parte del sistema interno documentado como agentes propios, se han implementado como scripts Python para poder demostrar sus

protocolos de forma realista. `Transportista.py` representa empresas de mensajería externas con perfiles configurables. Cada transportista calcula ofertas a partir de parámetros como precio base, factor de coste, precio mínimo y paso de concesión. Durante la negociación puede aceptar, rechazar o bajar el precio, y el sistema detecta cuándo un transportista ya no rebaja más su oferta.

`EmpresaVendedora.py` representa tiendas externas. Estas empresas pueden registrar productos nuevos en el Catalogador mediante un temporizador de demo y pueden recibir ventas delegadas cuando el producto es totalmente externo. Sus datos bancarios se guardan en el Tesorero para poder liquidar pagos. La configuración de estos actores se concentra en `external_agents_config.json`, lo que permite cambiar precios de transportistas, productos externos e IBANs sin modificar el código principal.

`EntidadBancaria.py` representa la pasarela bancaria externa. Para la práctica se ha configurado de forma determinista, pero se ha mantenido como servicio separado para conservar el protocolo de solicitud y confirmación de transferencia. Esta separación hace visible que los cobros, pagos y devoluciones no son operaciones locales de Ventas o Logística, sino acciones delegadas al agente financiero y al actor bancario.

5.5 Protocolos, trazabilidad y temporizadores

Los protocolos definidos en el diseño se han trasladado a la implementación mediante mensajes RDF con performativas FIPA. Se usan `request` para peticiones de acción, `inform` para resultados positivos, `refuse` para rechazos, `query-if` para comprobar disponibilidad de productos en centros logísticos y `query-ref` para solicitar datos como compras históricas o valoraciones. Cuando un protocolo necesita un concepto específico que no era suficiente con una respuesta genérica, se ha añadido una clase RDF concreta, por ejemplo `ResultadoDevolucion`.

Para facilitar la depuración se implementó un agente `Logger.py`. Todos los agentes descubren el `Logger` mediante el Directorio y envían trazas de los paquetes intercambiados. La interfaz del logger normaliza los nombres de agentes y permite distinguir instancias concretas, por ejemplo varios centros logísticos o varios transportistas. Además, se filtran respuestas técnicas genéricas cuando no forman parte de un protocolo real, evitando que la visualización muestre falsos mensajes de respuesta.

Algunas percepciones del diseño son temporizadores. En lugar de depender de un cron real durante la demo, se exponen endpoints manuales como `/tick/envios`, `/tick/feedback`, `/tick/recomendaciones` y `/tick/nuevo-producto`. El script `demo_tick.sh` ejecuta estos temporizadores de forma controlada para acelerar el envío de lotes, la petición de feedback, la generación de recomendaciones y el registro de productos externos. Los tiempos relevantes también son configurables con variables o argumentos, como el margen para pedir feedback o la ventana de urgencia de un lote.

5.6 Justificación de las decisiones técnicas

La elección de Flask se debe a que permite construir agentes independientes con muy poca infraestructura adicional, manteniendo clara la frontera entre procesos. La comunicación HTTP con RDF serializado simplifica la ejecución local y conserva, al mismo tiempo, una representación semántica de los mensajes. El uso de un módulo compartido para RDF/FIPA evita errores repetidos y hace que cualquier cambio de protocolo se aplique de forma homogénea.

La persistencia se ha mantenido en memoria porque el objetivo de la práctica es demostrar el comportamiento multiagente, no desplegar una tienda real con una base de datos transaccional. Aun así, cada agente mantiene sus propios recursos de creencias: catálogo, stock, lotes, compras, devoluciones, pagos, valoraciones, negociaciones o transferencias. Para hacer estos estados visibles, todos los agentes importantes ofrecen un endpoint `/info` con sus datos runtime.

Por último, la implementación separa claramente los flujos síncronos y los proactivos. La búsqueda y confirmación de compra son inmediatas desde la perspectiva del usuario, mientras que el envío, el cobro final de lotes, las recomendaciones y el feedback aparecen más tarde mediante temporizadores. Esta decisión reproduce mejor el modelo del diseño y evita mezclar en una única operación todo el ciclo de vida de la compra.

6. Descripción detallada de la planificación del trabajo

Para la planificación de este proyecto, nuestro principio fundamental desde el primer día fue el realismo. Quisimos alejarnos por completo de cronogramas teóricos que luego no se ajustaran ni a la carga de trabajo real ni a las dinámicas de nuestro equipo. Con este enfoque en mente, decidimos adoptar una metodología basada en los principios Agile, pero adaptando sus ceremonias a nuestra disponibilidad horaria y al calendario académico.

El eje central que utilizamos para la sincronización del grupo fueron las reuniones semanales, las cuales hicimos coincidir de forma estratégica con las clases de laboratorio. Estos encuentros presenciales nos sirvieron como sesiones de revisión y planificación (muy similares a los Sprint Planning y Sprint Review del entorno Agile). En ellas evaluábamos el progreso de la semana anterior, debatíamos el enfoque técnico de las soluciones que íbamos a implementar y nos repartíamos las nuevas tareas entre los miembros del grupo, fijando siempre objetivos claros a corto plazo.

6.1 Canales de Comunicación

Para garantizar que todos tuviéramos un acceso inmediato a la información y mantener la total transparencia en el desarrollo, establecimos canales específicos según el tipo de interacción que necesitábamos:

Gestión de Ficheros y Código: Aquí nos centramos en utilizar un repositorio centralizado en Git como único punto de verdad para compartir y almacenar absolutamente todos los archivos del proyecto. Esto nos aseguró mantener un control estricto y una total trazabilidad de los cambios realizados.

Comunicación: Más allá de vernos en las sesiones presenciales del laboratorio, nos organizamos para hacer reuniones virtuales los martes a través de Discord. Estas sesiones las convocábamos siempre que fuera necesario, pero tuvimos que recurrir a ellas con especial intensidad principalmente durante la fase en la que trabajamos con el programa Protégé, ya que nos sirvieron para discutir enfoques técnicos, resolver dudas emergentes en caliente y evitar que cualquiera de nosotros se quedara bloqueado.

6.2 Planificación y Estrategia de Desarrollo por Componentes

6.2.1. Fase de Diseño con Prometheus Design Tool

Mientras utilizábamos esta herramienta, nos topamos con un inconveniente técnico importante: nos costaba muchísimo hacer el trabajo en paralelo de forma eficiente a través de Git. La naturaleza de los ficheros que genera Prometheus dificultaba enormemente la resolución de conflictos al intentar fusionar archivos simultáneamente. Ante este problema, optamos de manera muy consciente por un enfoque puramente secuencial. Lo que hacíamos en las reuniones semanales de laboratorio era fragmentar el bloque de trabajo de Prometheus en apartados muy específicos y nos asignábamos fechas límite consecutivas entre nosotros. De esta manera, cada uno trabajaba de forma secuencial sobre la última versión limpia que había consolidado el compañero anterior.

6.2.2. Fase de Protégé

Para el modelado de la ontología sabíamos que necesitábamos un desarrollo altamente colaborativo y simultáneo. Para evitar los problemas de versiones que tuvimos con Prometheus, decidimos dar el salto a la plataforma WebProtégé. Al tratarse de un entorno en la nube, nos permitió trabajar de forma completamente paralela y simultánea sobre el mismo modelo sin pisarnos el trabajo. Como mencionamos antes, coordinamos este diseño apoyándonos en las llamadas síncronas de Discord de los martes.

6.2.3. Desarrollo del Código Python

Para la implementación de los agentes en Python seguimos una estrategia puramente paralela utilizando Git. Cada miembro del grupo asumió la responsabilidad de desarrollar funciones, agentes o módulos totalmente independientes. Las integraciones de código las fuimos subiendo de manera continua al repositorio, resolviendo los conflictos de versiones que surgían a través de la propia plataforma de Git de forma muy fluida.

7. Evaluación crítica de los resultados

La adopción del paradigma de Sistemas Multiagente mediante una arquitectura de servicios desacoplados en Flask ha demostrado ser una decisión acertada. La separación explícita de responsabilidades en agentes independientes (Catálogo, Ventas, Logística y Tesorería) garantiza una alta cohesión en el sistema y simplifica el mantenimiento del código.

Un gran acierto del diseño es la gestión asíncrona de los pedidos. El Agente Ventas puede procesar la compra del usuario de forma inmediata sin verse bloqueado por la compleja asignación de rutas o transportistas; esta tarea se delega al Agente Logístico, que la gestiona en segundo plano mediante un temporizador.

El principal inconveniente de esta descentralización es la complejidad en la comunicación. El constante intercambio de mensajes estructurados añade una carga adicional a la red y exige un control muy estricto para que los estados internos de cada agente se mantengan sincronizados sin duplicar información.

7.1. Consideraciones sobre los límites de la solución y áreas de mejora

Aunque el prototipo cumple con todos los requisitos, se han identificado limitaciones estructurales que deberían corregirse para un despliegue en un entorno real:

7.1.1. Gestión de fallos en transacciones distribuidas

El sistema asume que las comunicaciones con la Entidad Bancaria externa siempre son exitosas. Si se produce un fallo de red o un error bancario justo después de haber modificado el stock en los almacenes, los datos del sistema quedarían en un estado inconsistente (productos reservados pero no cobrados). Para solucionar esto, sería necesario diseñar un protocolo de confirmación en dos fases que asegure que la compra y el pago se consolidan como una única operación indivisible.

7.1.2. Centralización del Directorio de Agentes

A pesar de que el sistema es modular, el Directorio de Agentes constituye un punto único de vulnerabilidad. Si este servicio de registro se interrumpe o se satura, los agentes logísticos no podrán descubrir a los transportistas y el Agente Ventas quedará incomunicado con los centros logísticos. Para garantizar la disponibilidad de la tienda, este directorio debería replicarse en varios servidores.

7.1.3. Escalabilidad del análisis de datos

El Agente Valorador genera recomendaciones proactivas leyendo de forma secuencial todo el historial de compras y búsquedas del usuario. Este enfoque funciona correctamente con un volumen de datos controlado, pero resultaría inviable en una tienda con miles de clientes. Para evitar cuellos de botella, este procesamiento de datos debería realizarse de forma desconectada del flujo principal o apoyarse en bases de datos optimizadas para relaciones complejas.

7.1.4. Criterio de asignación logística elemental

La fragmentación de pedidos entre diferentes centros logísticos se realiza mediante una simple comprobación matemática de existencias. El diseño actual no incluye criterios de proximidad geográfica entre el almacén y el cliente. Para optimizar los costes de envío, el algoritmo del Agente Ventas debería priorizar el centro logístico más cercano a la dirección de entrega antes de comprobar el resto de almacenes.

7.2. Elementos incluidos del nivel avanzado.

En el desarrollo de esta práctica se ha optado por realizar el nivel avanzado completo, implementando de forma íntegra las 5 tareas de nota extra en lugar del mínimo de 3 exigido. La integración de estos elementos adicionales ha modelado de forma transversal la arquitectura de agentes, los escenarios de interacción, el diseño conceptual de la ontología y el almacenamiento del sistema.

A continuación se detalla qué elementos se han incluido y cómo se han implementado en el diseño:

1. Agentes de transporte dinámicos (Servicio de Registro y Descubrimiento)

Qué se ha incluido: Un mecanismo por el cual las agencias de transporte no están prefijadas ni acopladas estáticamente en el código del sistema. En su lugar, se contempla un entorno dinámico donde los centros logísticos descubren en tiempo de ejecución con qué empresas pueden contactar en cada momento.

Cómo se ha implementado: El Agente Logístico hace uso del Directorio de Agentes (DF) de la plataforma para registrar y buscar los servicios de transporte disponibles bajo demanda. Al activarse el plan de envío, el Agente Logístico realiza una consulta dinámica al Directorio de Agentes para obtener las direcciones actualizadas de los agentes de transporte activos en ese instante y dentro del área geográfica correspondiente.

2. Negociación compleja de envío (Contraofertas)

Qué se ha incluido: Un protocolo de interacción bidireccional e iterativo entre el centro logístico y los transportistas orientada a la optimización de costes, sustituyendo la asignación directa por un flujo de negociación.

Cómo se ha implementado: Integrado dentro de la capacidad de gestión de transportistas del Agente Logístico. Al agrupar un lote de envío, el plan correspondiente solicita un presupuesto a los agentes de transporte descubiertos. Tras recibir y analizar las ofertas iniciales, el plan calcula una contraoferta (con un precio ligeramente inferior a la oferta mínima recibida) y la envía a los transportistas. Estos responden de forma autónoma (aceptando, proponiendo una rebaja intermedia o rechazando). Finalmente, el algoritmo de adjudicación del Agente Logístico asigna el envío a la respuesta más económica o, en caso de que nadie acepte la contraoferta, a la primera oferta más barata (resolviendo empates al azar).

3. Pedidos multi-centro logístico y distribución geográfica

Qué se ha incluido: La capacidad de procesar pedidos combinados que contienen productos almacenados físicamente en diferentes ubicaciones geográficas o almacenes, coordinando su distribución independiente sin penalizar la experiencia de compra centralizada del cliente.

Cómo se ha implementado: Este criterio motivó la separación arquitectónica entre los escenarios de venta y logística. El Agente Ventas recibe el pedido global del cliente, consulta la disponibilidad distribuida, fragmenta el pedido según el stock de cada almacén y añade cada artículo de forma asíncrona a los lotes pendientes de sus respectivos centros. Posteriormente, cada Agente Logístico (representando un centro geográfico diferente) gestiona su propia negociación con sus transportistas locales. Como consecuencia, la ontología y el flujo contemplan que el usuario reciba múltiples notificaciones independientes con diferentes fechas de entrega y transportistas asociados para un mismo pedido inicial.

4. Gestión financiera y pasarela de pagos (Agente Tesorero)

Qué se ha incluido: La centralización y securización de todos los flujos monetarios y transacciones bancarias del ecosistema de la tienda virtual, gestionando cobros a clientes, reembolsos por devoluciones y la liquidación periódica a vendedores externos.

Cómo se ha implementado: Se diseñó y encapsuló de forma aislada el Agente Tesorero (asumiendo el Rol Financiero del sistema). Este agente interactúa de manera dedicada con la base de datos Pagos en curso / Registro de pagos y con la pasarela bancaria externa. El Agente Ventas delega de forma asíncrona los flujos económicos en el Tesorero mediante

mensajes específicos: quiero cobrar al usuario ante una compra, cobrar-envios-lote para el coste logístico, quiero-devolver en caso de desistimiento, y mediante planes temporizados independientes para liquidar los saldos acumulados de la tienda hacia los vendedores externos de la plataforma.

5. Solicitud de Valoraciones y Recomendación Proactiva de Productos

Qué se ha incluido: Un componente proactivo basado en eventos temporales diseñado para capturar el feedback del cliente tras la entrega de los productos y para dinamizar las ventas mediante un motor de recomendaciones personalizadas basadas en el comportamiento histórico.

Cómo se ha implementado: Se ha diseñado el Agente Valorador, el cual opera de forma proactiva mediante temporizadores (*timers*):

- **Captura de Feedback:** Un temporizador periódico despierta al agente para solicitar al Agente Ventas el listado de pedidos entregados con una antigüedad determinada. El Agente Valorador envía de forma proactiva una petición de valoración al cliente y almacena las puntuaciones y reseñas recibidas en la base de datos compartida de Valoraciones.
- **Recomendación Proactiva:** Otro temporizador independiente activa el plan de recomendación. El agente consulta a Ventas el histórico de compras y búsquedas del usuario, procesa esta información cruzándola con las tendencias y valoraciones de la base de datos de Valoraciones, genera sugerencias de catálogo altamente personalizadas y las envía de forma autónoma al cliente.